Федеральное государственное образовательное бюджетное учреждение
высшего образования
«ФИНАНСОВЫЙ УНИВЕРСИТЕТ ПРИ ПРАВИТЕЛЬСТВЕ РОССИЙСКОЙ ФЕДЕРАЦИИ»

Факультет: ИТИАБД

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовой работе по дисциплине
«Современные технологии программирования»

на тему:

«Разработка информационной системы ломбарда с ролевой моделью доступа»

Выполнил:
студент группы ТРПО24-3
Сурков Иван Сергеевич

Руководитель:
Цыренов Доржо Гунденович

Москва, 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
1 ОБЪЕКТ И ПРЕДМЕТ ИССЛЕДОВАНИЯ 3
2 ПОСТАНОВКА ЗАДАЧИ 4
3 ОПИСАНИЕ ПРОГРАММЫ 6
3.1 Архитектура приложения 6
3.2 Модель данных 7
3.3 Алгоритмические решения 9
3.4 Безопасность 11
3.5 Обоснование выбора технологий и сравнительный анализ 13
3.6 Тестирование системы 14
4 ОПИСАНИЕ ИНТЕРФЕЙСА ПРОГРАММЫ 15
4.1 Страницы аутентификации 15
4.2 Главная страница 16
4.3 Раздел «Мои займы» 17
4.4 Раздел «Создать заявку» 18
4.5 Раздел «Мой счет» 18
4.6 Раздел «Магазин» и «Корзина» 19
4.7 Раздел «Мои заказы» 20
4.8 Раздел «Уведомления» 21
4.9 Раздел «Мой профиль» 21
4.10 Раздел «Все займы» (для администратора и менеджера) 22
4.11 Раздел «Оценка и одобрение заявки» 22
4.12 Раздел «Прием вещей» 23
4.13 Раздел «Залоги» 23
4.14 Раздел «Клиенты» 24
4.15 Раздел «Заказы» 25
4.16 Раздел «Склад» 26
4.17 Раздел «Управление магазином» 27
4.18 Раздел «Пользователи» (только администратор) 27
4.19 Раздел «Статистика» (только администратор) 27
4.20 Раздел «Об авторе» 28
4.21 Страницы ошибок 28
5 СТРУКТУРА РЕАЛИЗОВАННЫХ КЛАССОВ И ИХ НАЗНАЧЕНИЕ 29
ЗАКЛЮЧЕНИЕ 32
СПИСОК ЛИТЕРАТУРЫ 32
ПРИЛОЖЕНИЯ 34
ПРИЛОЖЕНИЕ А. Класс SecurityConfig 34
ПРИЛОЖЕНИЕ Б. Класс LoanController 36
ПРИЛОЖЕНИЕ В. Класс LoanService 43
ПРИЛОЖЕНИЕ Г. Класс GlobalExceptionHandler 47
ПРИЛОЖЕНИЕ Д. Класс Loan (Модель данных) 48
ПРИЛОЖЕНИЕ Е. Файл application.properties 49

ВВЕДЕНИЕ

Ломбард выдает краткосрочные займы под залог имущества. Если заем вовремя не возвращен, вещь остается у ломбарда и выставляется на продажу. По каждой сделке нужно хранить сведения о клиенте, о вещи и ее оценке, о сумме, ставке и сроке возврата, а также следить за просрочками. При ручном учете в таких данных неизбежны ошибки и расхождения. Кроме того, клиенты все чаще ожидают, что заявку можно подать и отследить удаленно, а невыкупленную вещь - купить онлайн. Поэтому автоматизация учета залогов и займов остается актуальной задачей: она снижает число ошибок, ускоряет обслуживание и сокращает количество визитов клиента в офис.

Цель курсовой работы - разработать веб-приложение для ломбарда с ролевой моделью доступа, в котором каждой роли доступен свой набор действий. Приложение должно охватывать весь цикл: заявку на заем, оценку и выдачу, погашение, учет невыкупленных вещей на складе и их продажу через встроенный магазин.

Для достижения цели решались следующие задачи: изучить бизнес-процессы ломбарда; распределить функции между администратором, менеджером и клиентом; выбрать технологии; спроектировать архитектуру и базу данных; реализовать серверную часть и интерфейс; обеспечить защиту и обработку ошибок; проверить результат вручную и автотестами.

1 ОБЪЕКТ И ПРЕДМЕТ ИССЛЕДОВАНИЯ

Объектом исследования служит повседневная работа ломбарда: прием вещей в залог и их оценка, выдача и возврат займов, хранение невыкупленных вещей и их продажа через магазин.

Предметом исследования служит то, как в ломбарде обрабатывается информация, и программа, которая это делает: учет клиентов, залогов, займов, заказов и товаров с разным доступом в зависимости от роли.

Для ломбардов уже есть готовые учетные программы, в том числе решения на платформе 1С и отдельные коммерческие продукты. Они закрывают кассу,

бухгалтерию и оформление залогового билета, но, как правило, платные, трудно дорабатываются под конкретный ломбард и не дают клиенту личного кабинета, а также магазина для продажи невыкупленных вещей. Именно эту нишу закрывает разработанное приложение: в нем заявку, погашение и покупку клиент оформляет сам, а сотрудники видят все в одном интерфейсе.

2 ПОСТАНОВКА ЗАДАЧИ

В системе три роли: администратор, менеджер и клиент. Без входа в аккаунт открываются только страницы входа и регистрации, все остальное доступно после аутентификации.

Что система умеет для каждой роли:

Клиент:

1) регистрация и авторизация в системе;

2) просмотр и правка своих данных (ФИО, email, телефон), смена пароля;

3) создание заявки на заем: желаемая сумма, срок, описание вещи в залог и фотографии;

4) просмотр списка своих займов с фильтрацией по статусу, поиском и сортировкой;

5) подтверждение или отклонение условий займа, предложенных менеджером;

6) погашение займа с личного счета;

7) пополнение и вывод средств со своего счета, просмотр истории операций;

8) магазин: просмотр товаров, корзина, оформление заказа (доставка или самовывоз);

9) просмотр своих заказов и их статусов;

10) просмотр уведомлений о событиях (одобрение займа, напоминания о сроке возврата, просрочка, статусы заказов и т.д.);

11) доступ к странице поддержки и странице «Об авторе».

Менеджер:

1) просмотр и обработка заявок на прием вещей (оценка залога, одобрение или отклонение займа);

2) редактирование данных залога (название, описание, оценка, фотографии);

3) управление займами: изменение статуса, продление срока, перевод на склад или продажу, удаление;

4) просмотр и редактирование базы клиентов, создание новых клиентов;

5) управление заказами: изменение статуса (оплачен, собран, доставлен, отменен);

6) управление складом: редактирование товаров, публикация в магазин, удаление;

7) управление магазином: снятие товаров с витрины;

8) просмотр уведомлений;

9) экспорт данных в CSV.

Администратор:

1) все функции менеджера;

2) управление пользователями: создание, смена роли, удаление;

3) просмотр статистики: число пользователей, клиентов и займов, средняя сумма займа, среднее время до принятия решения, средний срок займа, финансовые показатели и диаграмма распределения займов по статусам;

4) просмотр профилей менеджеров со статистикой их действий;

5) импорт пользователей из CSV-файла.

Общие требования:

1) отображение данных предметной области в табличной форме с возможностью просмотра всех записей;

2) реализация операций создания, редактирования и удаления объектов (CRUD);

3) поиск по всем записям предметной области;

4) сортировка по алфавиту, дате, числам и статусу;

5) устойчивость к ошибкам: при неверном вводе, недопустимых значениях, пустых данных или сбое базы приложение не падает, а показывает сообщение (всплывающее уведомление или страницы 403, 404, 500);

6) защита от SQL-инъекций и XSS;

7) разграничение доступа на основе ролей.

Принятые ограничения:

1) приложение сделано как веб-приложение: страницы подстраиваются под ширину экрана с помощью медиазапросов CSS, но отдельная мобильная версия не разрабатывалась, так как условия дисциплины ее не требуют;

2) в качестве СУБД используется реляционная база PostgreSQL;

3) аутентификация и авторизация реализованы на основе Spring Security;

4) для хранения фотографий залогов используется файловая система сервера, в базе данных хранятся пути к файлам;

5) система рассчитана на работу в локальной сети или в сети Интернет при соответствующей настройке;

6) номера всех объектов (пользователи, клиенты, займы, залоги, заказы) выдаются из таблицы number_sequences;

7) пополнение и вывод средств имитируются, платежная система не подключена.

3 ОПИСАНИЕ ПРОГРАММЫ

3.1 Архитектура приложения

Приложение разбито на три слоя, и каждый отвечает за свое:

1) **Слой клиента (представления)** - то, что пользователь видит в браузере. Страницы строятся через Thymeleaf, оформление написано на CSS (пять файлов: base, layout, components, pages и theme, в том числе светлая и темная темы), плюс небольшие сценарии на JavaScript. Данные сюда приходят уже готовыми от сервера, бизнес-логики и доступа к базе в этом слое нет.

2) **Слой сервера (бизнес-логики)** - код на Java (Spring Boot). Контроллеры принимают HTTP-запросы, сервисы содержат бизнес-логику, права проверяет

Spring Security. В ответ сервер возвращает готовую HTML-страницу либо делает редирект.

3) **Слой данных** - сущности, репозитории и сама PostgreSQL. К базе обращаются через Spring Data JPA и Hibernate: репозитории - это интерфейсы, через которые сервисы читают и пишут данные.

Путь запроса такой: браузер => HTTP-запрос => контроллер => сервис => репозиторий => база данных. Ответ идет в обратную сторону: сервис => контроллер => HTML-страница => браузер. Такое разделение упростило тесты: сервисы проверяются с заглушками, контроллеры - без базы.

Структура проекта организована по пакетам:

1) com.lombard.controller - контроллеры, обрабатывающие HTTP-запросы;

2) com.lombard.service - сервисы с бизнес-логикой;

3) com.lombard.repository - репозитории для доступа к данным;

4) com.lombard.model - JPA-сущности предметной области;

5) com.lombard.config - конфигурация приложения (безопасность, веб-конфигурация, инициализация данных, глобальные атрибуты модели);

6) com.lombard.util - вспомогательные утилиты (экспорт в CSV, откат транзакции при обработанной ошибке).

Настройки вынесены в файл application.properties (приложение Е), он приложен в архиве с проектом.



Скриншот 1 (Схема трехзвенной архитектуры)

3.2 Модель данных

Данные хранятся в реляционной базе PostgreSQL. В системе используются такие сущности:

1) **User** - пользователь системы (логин, пароль, роль, уникальный номер);

2) **Client** - клиент ломбарда (ФИО, телефон, email, номер клиента, связь с пользователем);

3) **Loan** - заем (номер, сумма, ставка, статус, даты создания и возврата, связь с клиентом);

4) **Collateral** - залоговое имущество (номер вещи, название, описание, оценка, фотографии, связь с займом);

5) **Order** - заказ товара из магазина (номер, сумма, статус, способ доставки, адрес, связь с клиентом);

6) **OrderItem** - позиция заказа (связь с заказом и займом, название товара, цена);

7) **Account** - лицевой счет клиента (баланс, связь с клиентом);

8) **AccountTransaction** - транзакция по счету (сумма, тип, описание, дата);

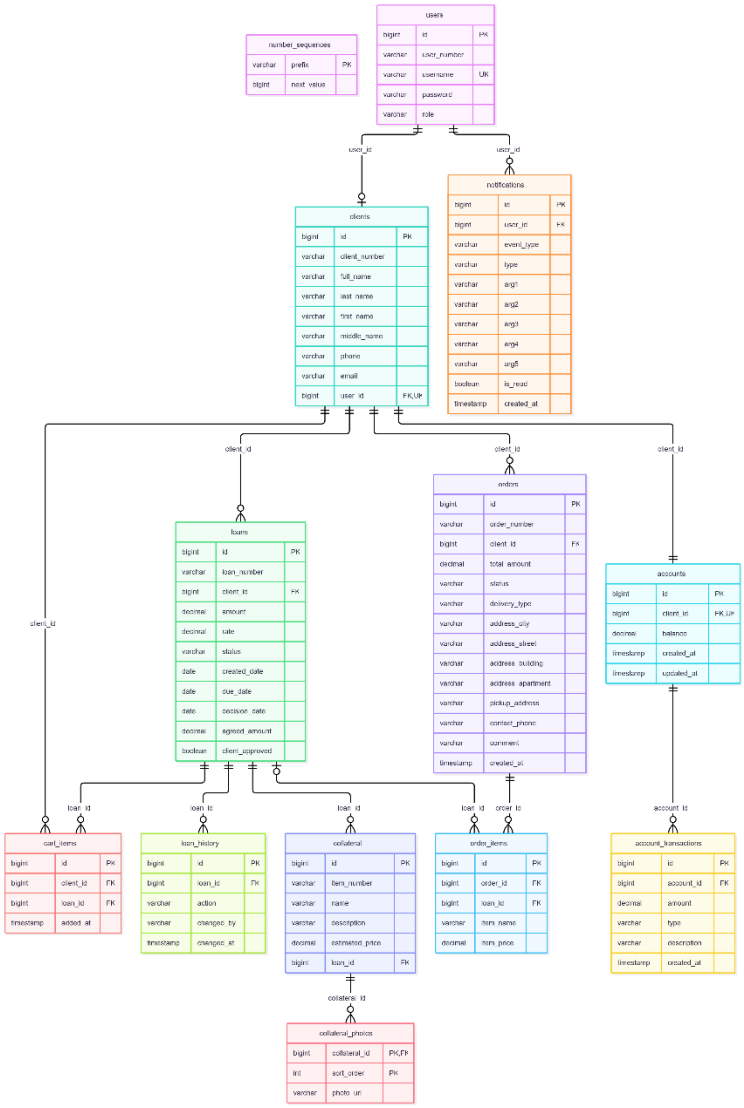
9) **CartItem** - элемент корзины (связь клиента и товара);

10) **Notification** - уведомление пользователя (тип события, параметры, статус прочтения);

11) **LoanHistory** - история изменений займа (действие, автор, дата);

12) **NumberSequence** - последовательность для генерации уникальных номеров.

Связи между сущностями такие: клиент и пользователь связаны один к одному (OneToOne); у клиента может быть много займов (OneToMany); в одном займе может быть несколько залогов; в заказе несколько позиций; у клиента один счет, но много транзакций по нему.



Скриншот 2 (ER-диаграмма базы данных)

3.3 Алгоритмические решения

3.3.1 Аутентификация и авторизация

Пароль проверяется через BCrypt. Когда вход прошел успешно, данные пользователя и его роль кладутся в сессию, и дальше Spring Security сверяет с ними каждый запрос по правилам из раздела 3.4.

3.3.2 Нумерация объектов

Номера берутся из таблицы number_sequences. NumberSequenceService в отдельной транзакции блокирует нужную строку, увеличивает счетчик и отдает номер в виде «префикс + число». Параллельный запрос в это время ждет снятия блокировки, поэтому два одинаковых номера получиться не могут.

Буква в номере показывает тип объекта: L - займы, C - клиенты, M - вещи, O - заказы; у пользователей буква зависит от роли (A, M или U).

3.3.3 Жизненный цикл займа

Заем проходит статусы «В ожидании» => «Ожидает подтверждения» => «Выдан» => «Закрыт» (или «Просрочен» => «На складе» => «На продаже» => «Куплен»). Допустимые переходы заданы картой ALLOWED_TRANSITIONS в LoanController, недопустимые блокируются с сообщением об ошибке, каждое изменение записывается в историю займа.

3.3.4 Оценка залога и одобрение займа

Менеджер вводит оценку вещи, сумму займа и ставку. Клиент должен вернуть сумму + сумма * ставка / 100, с округлением до двух знаков. Заем уходит в статус «Ожидает подтверждения», клиенту приходит уведомление. Принял условия - статус «Выдан», отказался - заявка отклонена.

3.3.5 Обработка просрочек

Просрочки проверяет метод LoanService.processOverdueLoans. Он запускается каждую ночь в 01:00 (класс OverdueLoanScheduler, аннотация @Scheduled) и при открытии менеджером списка займов. За 3 дня, за 1 день и в последний день срока клиенту приходит напоминание; после срока заем получает статус «Просрочен» и уведомление уходит клиенту и сотрудникам; при просрочке от 30 дней вещь переводится на склад. Повторные уведомления отсекаются проверкой истории займа (LoanHistory).

3.3.6 Погашение займа

Система проверяет наличие достаточной суммы на лицевом счете клиента, списывает сумму к возврату, переводит заем в статус «Закрыт» и фиксирует транзакцию по счету. Строка счета при этом блокируется (PESSIMISTIC_WRITE), чтобы две одновременные операции не исказили баланс.

3.3.7 Оформление заказа

Заказ оформляется из корзины, клиент выбирает доставку или самовывоз. Сначала система проверяет баланс счета, и если денег хватает, сумма списывается и создается заказ с позициями. Займы по купленным вещам переходят в статус «Куплен», корзина пустеет, а клиент и сотрудники получают уведомления.

Списание денег и создание заказа выполняются в одной транзакции: если на каком-то шаге произошла ошибка (например, товар уже купил другой клиент), в блоке catch вызывается TransactionUtils.rollbackCurrent() и транзакция откатывается целиком - деньги остаются на счете. Так же устроена регистрация: без профиля клиента не сохраняется и учетная запись.

3.3.8 Как работают поиск и сортировка

Поиск по клиентам и залогам написан в репозиториях на JPQL с параметром @Param, регистр не учитывается (LOWER и LIKE). Займы фильтруются в памяти потоком Stream в методе LoanService.searchLoans: по статусу и по строке поиска, которая сравнивается с номером займа, ФИО и номером клиента и номером вещи. Сортировка делается в контроллерах через Comparator (по алфавиту, дате, числам и статусу).

3.3.9 Экспорт данных в CSV

CsvExporter собирает CSV-файл с разделителем «;». Значения, которые начинаются с символов =, +, - или @, он экранирует, чтобы Excel не принял их за формулы.

3.4 Безопасность

3.4.1 Вход в систему и хранение паролей

В базе пароли лежат только в виде хешей, которые строятся алгоритмом BCrypt с солью, открытый пароль нигде не сохраняется и не пересылается. При входе пароль проверяется методом matches класса BCryptPasswordEncoder.

3.4.2 Авторизация и разграничение доступа

Разграничение доступа сделано на Spring Security в два уровня. Первый - правила для URL-адресов в классе SecurityConfig (например, /admin/users доступен только администратору, /loans - администратору и менеджеру). Второй - аннотация @PreAuthorize на классах AdminClientController, AdminOrderController, AdminManagerController, IntakeController, WarehouseController и на методах управления магазином в ShopController: даже если в правилах URL что-то пропущено, метод не выполнится без нужной роли. Фотографии залогов (/uploads/**) отдаются только вошедшим пользователям. Администратор не может менять роль и удалять самого себя, товары со склада удаляет только администратор. При нехватке прав открывается страница 403.

3.4.3 Защита от CSRF-атак

CSRF-защита включена. Токен хранится в cookie XSRF-TOKEN, Thymeleaf сам добавляет скрытое поле _csrf в формы с th:action. POST-запрос без верного токена отклоняется с кодом 403, что закрывает типовую подделку запроса со стороннего сайта.

3.4.4 SQL-инъекции

Запросы к БД идут через Hibernate с параметрами, в том числе именованными (@Param). Пользовательский ввод напрямую в SQL не подставляется, поэтому внедрить в запрос свой код нельзя.

3.4.5 XSS

Thymeleaf сам экранирует все, что выводится на страницу, так что скрипт, вставленный в поле формы, просто отобразится текстом. При экспорте в CSV значения дополнительно проходят через CsvExporter.esc: она нейтрализует формулы Excel и спецсимволы.

3.4.6 Валидация ввода

Все формы снабжены клиентской (HTML5-атрибуты required, pattern, minlength, maxlength) и серверной валидацией (аннотации @NotBlank, @Pattern,

@Email, @DecimalMin, проверки в контроллерах и сервисах). Некорректные данные отклоняются с выводом информативного сообщения. Загружаемые фотографии проверяются отдельно: допускаются только JPG и PNG до 5 МБ, содержимое файла читается как изображение (ImageIO), имя файла генерируется заново через UUID, на один залог не больше 10 фото. При импорте пользователей из CSV проверяются число полей, длина пароля (не меньше 6 символов) и допустимость роли.

3.4.7 Ошибки

Все необработанные исключения попадают в класс GlobalExceptionHandler, и для каждого вида ошибки возвращается своя страница с правильным HTTP-кодом: 404 - страница не найдена, 403 - не хватает прав (в том числе когда сработал @PreAuthorize), 400 - неверный ввод, 409 - недопустимое действие или нарушение целостности данных, 500 - любой другой сбой. В последнем случае пользователь видит нейтральное сообщение, а подробности пишутся только в журнал. Правильные коды важны и для автотестов: тест, ожидающий успешный ответ, не пройдет, если на самом деле открылась страница ошибки. Работу обработчика проверяет класс GlobalExceptionHandlerTest.

3.5 Обоснование выбора технологий и сравнительный анализ

Java и Spring Boot выбраны потому, что на этом стеке построена дисциплина, а в проекте много денежной логики, где важна строгая типизация (суммы хранятся в BigDecimal). Для Node.js пришлось бы отдельно подбирать библиотеки для ролей, транзакций и ORM, в Spring Boot все это подключается стартерами. Минус - долгий запуск и больший расход памяти, для ломбарда некритичные.

Для доступа к данным сравнивались JDBC, MyBatis и Hibernate через Spring Data JPA. С JDBC пришлось бы вручную писать SQL и разбирать ResultSet для двенадцати сущностей; MyBatis хорош для сложных запросов, а здесь почти все запросы простые. Spring Data JPA строит их по имени метода

(findByClientId), а блокировка строки счета задается одной аннотацией @Lock. Обратная сторона - риск проблемы N+1 при неудачной настройке связей.

Интерфейс сделан на Thymeleaf, а не на React или Vue.js: отдельное SPA потребовало бы REST API, авторизации по токенам и второго проекта на JavaScript. В Thymeleaf один шаблон обслуживает все роли, недоступные кнопки просто не выводятся (th:if, sec:authorize). Цена - большинство действий перезагружает страницу.

Для двух гистограмм статистики хватило Chart.js: он подключается одним скриптом. Highcharts требует лицензию для коммерческого использования, а D3.js для обычной гистограммы избыточен.

Из двух СУБД, разрешенных методическими указаниями (MySQL и PostgreSQL), выбрана PostgreSQL: тип numeric хранит денежные суммы без потери точности, а блокировка SELECT ... FOR UPDATE, на которой построены операции со счетом, работает в ней предсказуемо.

3.6 Тестирование системы

В ходе тестирования системы были проверены следующие сценарии:

1) регистрация и авторизация пользователей с разными ролями (клиент, менеджер, администратор), при этом проверена валидация ввода (неверный формат телефона, пустые поля);

2) создание заявки на заем клиентом, ее одобрение менеджером и подтверждение клиентом, при этом проверено, что при одобрении клиенту приходит уведомление с условиями;

3) погашение займа с лицевого счета клиента, при этом проверено, что при недостатке средств выводится соответствующая ошибка;

4) оформление заказа в магазине: добавление в корзину, оплата с лицевого счета, смена статуса заказа администратором, при этом проверено, что клиенту приходит уведомление о доставке;

5) обработка ошибок: проверка вывода страниц 403 (Доступ запрещен) и 404 (Страница не найдена);

6) проверка разграничения прав доступа (клиент не может зайти в админку, менеджер не может удалить пользователя).

Помимо ручной проверки, написано 459 автоматических тестов на JUnit 5. Для каждого класса программы есть свой тестовый класс в том же пакете:

1) модульные тесты сервисов (114) - бизнес-логика проверяется без базы, репозитории заменены заглушками Mockito;

2) тесты веб-слоя (159, @WebMvcTest) - запросы через MockMvc с проверкой кода ответа, страницы, перенаправлений и прав разных ролей;

3) интеграционные тесты (127, @SpringBootTest) - полный запуск на встроенной базе H2, результат проверяется прямо в базе, каждый тест откатывается после выполнения;

4) остальные 59: модели (29), формы (20), репозитории (7), CSV (3).

Имена тестов составлены по шаблону «метод_условие_результат» (например, withdraw_notEnoughMoney_throwsException). Покрытие кода по данным JaCoCo - 83 % инструкций (сервисы - 95 %, контроллеры - 78 %); покрытие считается по всему коду, без исключений. В pom.xml задано правило: при покрытии ниже 80 % сборка mvn verify завершается ошибкой. Сейчас сборка проходит успешно (BUILD SUCCESS).

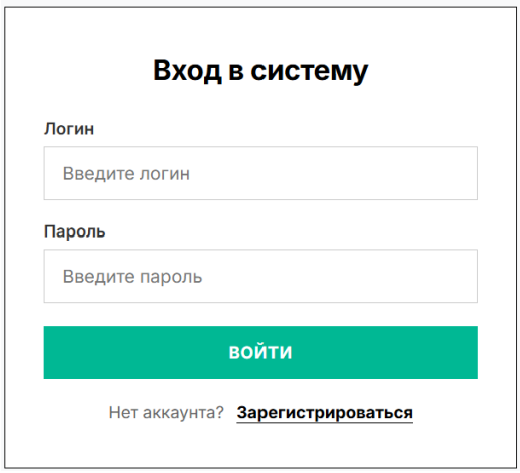
Тесты помогли найти и исправить несколько ошибок: при редактировании клиента администратором новое ФИО затиралось старым, на базе H2 падал расчет среднего времени ожидания, можно было отправить заявку без названия вещи. На каждую из них добавлен тест, который не даст ошибке вернуться.

4 ОПИСАНИЕ ИНТЕРФЕЙСА ПРОГРАММЫ

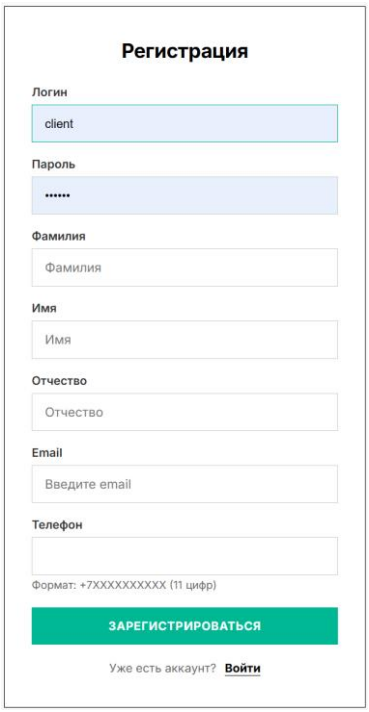
Интерфейс реализован в виде веб-приложения для трех ролей: администратор, менеджер и клиент. Используется единый стиль оформления, доступ к функциям разграничивается по роли; поддерживаются светлая и темная темы, русский и английский языки.

4.1 Страницы аутентификации

При запуске неавторизованный пользователь перенаправляется на страницу входа (/login) с полями логина и пароля, кнопкой входа и ссылкой на регистрацию; при неверных данных выводится сообщение об ошибке. Страница регистрации (/register) позволяет новому клиенту создать учетную запись, указав логин, пароль, ФИО, email и телефон в формате +7XXXXXXXXXX.



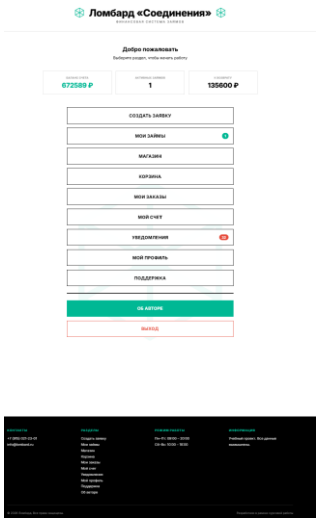
Скриншот 3 (Страница входа в систему)



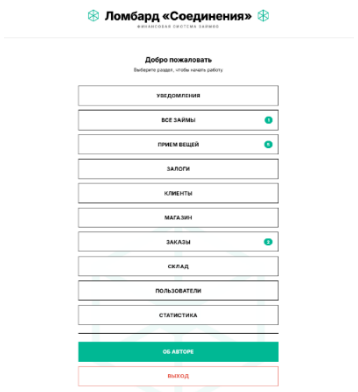
Скриншот 4 (Страница регистрации нового клиента)

4.2 Главная страница

После входа пользователь попадает на главную страницу (/), состав которой зависит от роли. Клиенту отображаются баланс лицевого счета, количество активных займов и общая сумма к возврату, администратору и менеджеру - плитки разделов системы.



Скриншот 5 (Главная страница для роли «Клиент»)



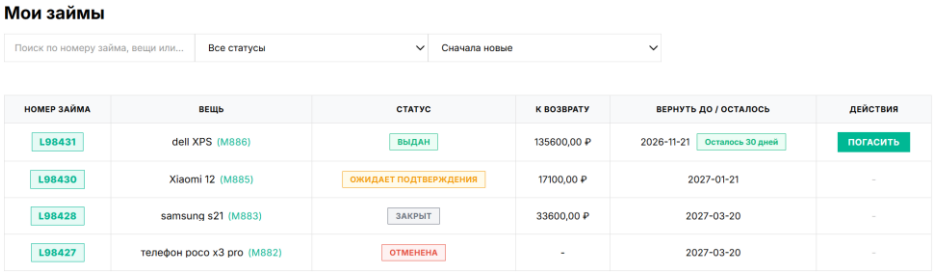


Скриншот 6 (Главная страница для роли «Администратор»)

4.3 Раздел «Мои займы»

Здесь клиент видит все свои займы: вещь, статус, сумму к возврату, дату возврата и таймер, который показывает, сколько дней осталось или на сколько

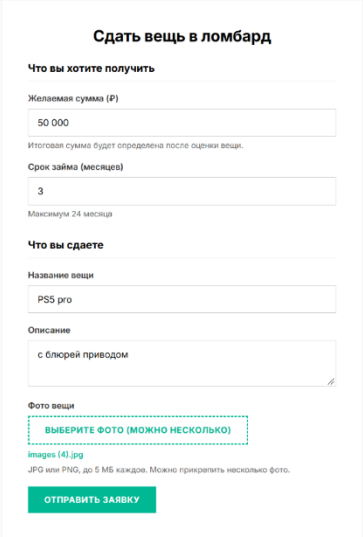
дней заем просрочен. Список можно отфильтровать по статусу, найти заем по номеру или вещи и отсортировать по сумме, дате или статусу. Из этого же раздела заем погашается, а заявку в статусе «В ожидании» можно отменить.



Скриншот 7 (Раздел «Мои займы» для роли «Клиент»)

4.4 Раздел «Создать заявку»

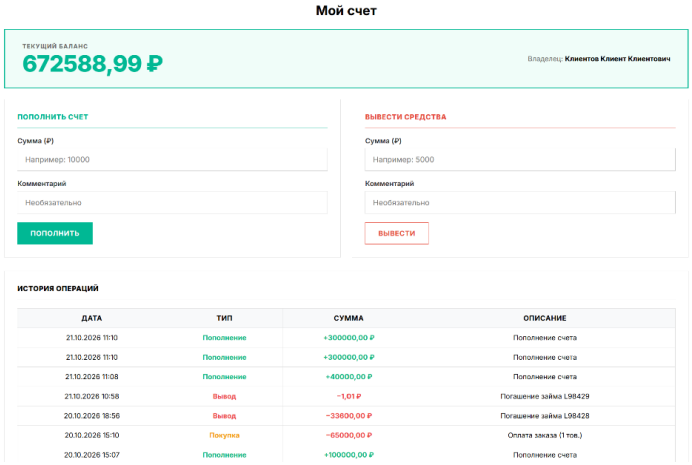
Клиент заполняет форму: желаемая сумма, срок займа (1-24 месяца), название и описание вещи, фотографии. После отправки заявка создается в статусе «В ожидании» и становится доступной менеджеру для оценки.



Скриншот 8 (Форма создания заявки на заем)

4.5 Раздел «Мой счет»

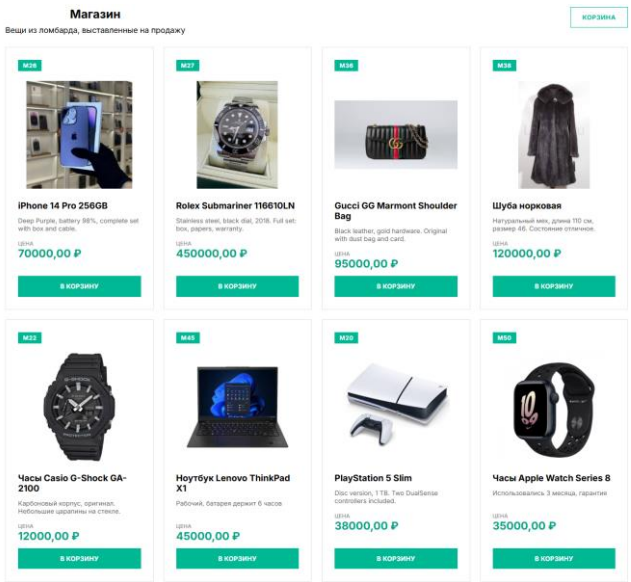
Страница показывает текущий баланс, формы пополнения и вывода и таблицу операций с датой, типом, суммой и описанием. Именно с этого счета списываются деньги при погашении займа и покупке в магазине.



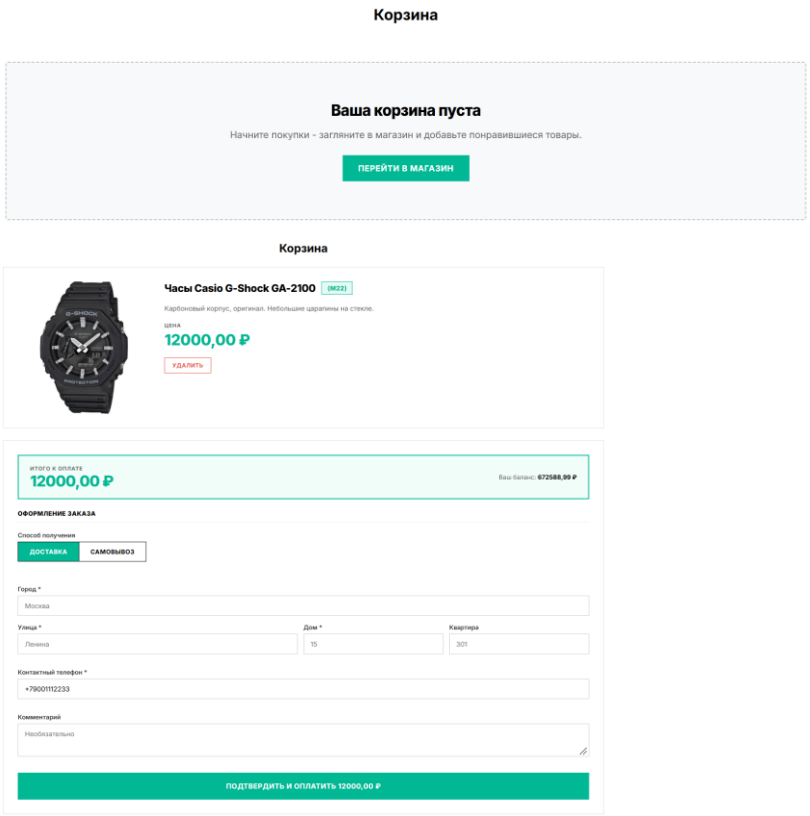
Скриншот 9 (Раздел «Мой счет» для роли «Клиент»)

4.6 Раздел «Магазин» и «Корзина»

В магазине товары, выставленные на продажу, представлены карточками с фотографией, названием, описанием и ценой. В корзине отображаются список товаров, итоговая сумма, баланс клиента и форма оформления заказа с выбором доставки или самовывоза. При нехватке средств выводится предупреждение со ссылкой на пополнение счета.



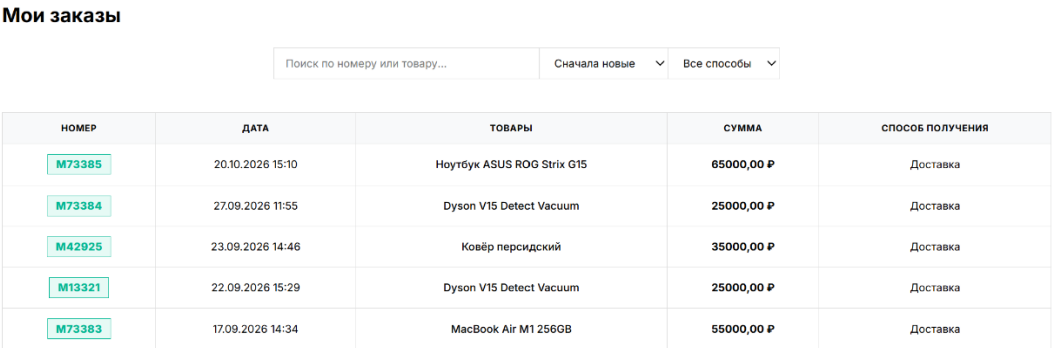
Скриншот 10 (Раздел «Магазин» для роли «Клиент»)



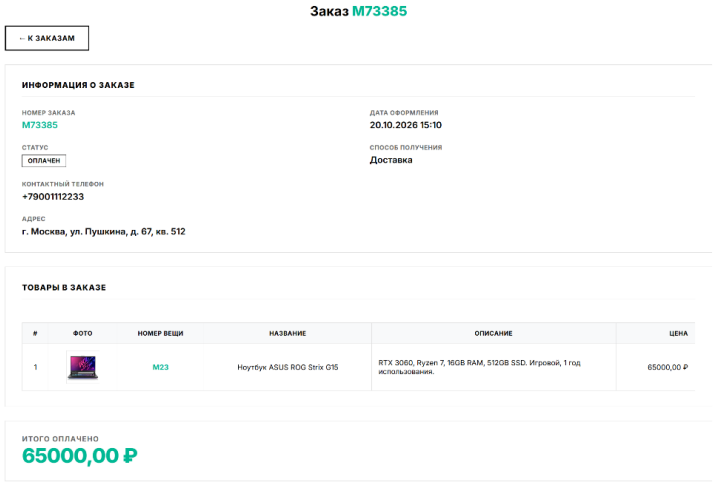
Скриншот 11 (Корзина и оформление заказа)

4.7 Раздел «Мои заказы»

В списке заказов есть поиск, фильтр по способу получения и сортировка. Открыв заказ, клиент видит его состав, сумму, текущий статус, адрес доставки или пункт самовывоза, телефон и комментарий.



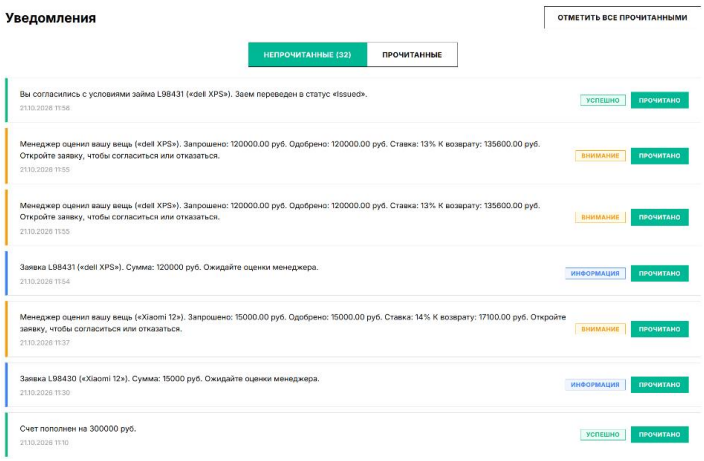
Скриншот 12 (Раздел «Мои заказы» для роли «Клиент»)



Скриншот 13 (Детали заказа клиента)

4.8 Раздел «Уведомления»

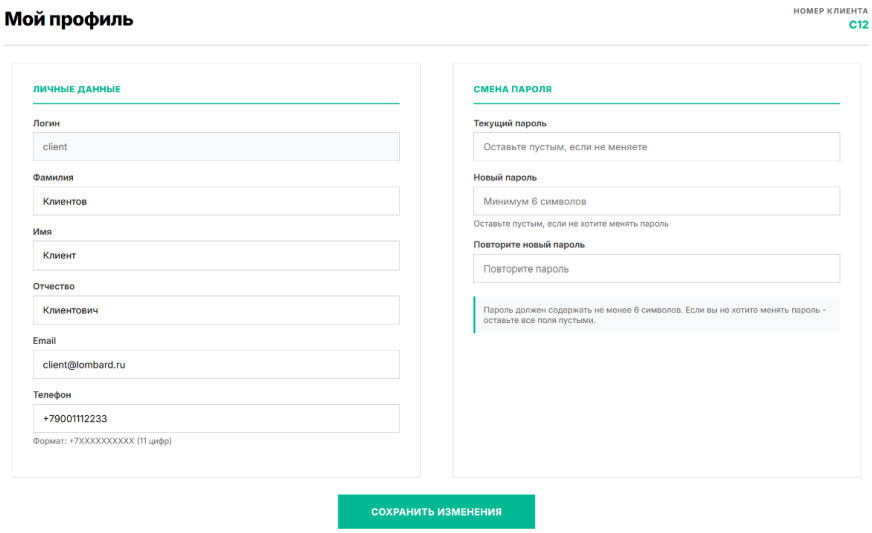
Сюда приходят сообщения об одобрении или отклонении займа, напоминания о сроке возврата, извещения о просрочке и смене статуса заказа. У клиента число непрочитанных показывается в боковом меню; отметить их прочитанными можно по одному или все сразу.



Скриншот 14 (Раздел «Уведомления» для роли «Клиент»)

4.9 Раздел «Мой профиль»

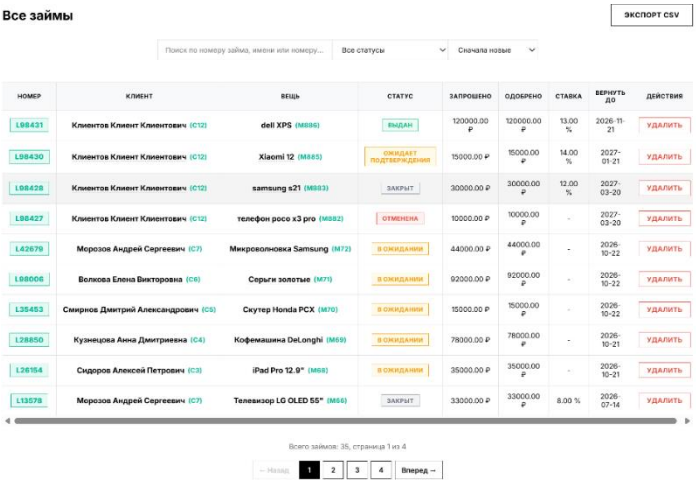
Клиент может просматривать и редактировать личные данные (фамилия, имя, отчество, email, телефон), а также менять пароль. Ввод данных контролируется регулярными выражениями.



Скриншот 15 (Раздел «Мой профиль» для роли «Клиент»)

4.10 Раздел «Все займы» (для администратора и менеджера)

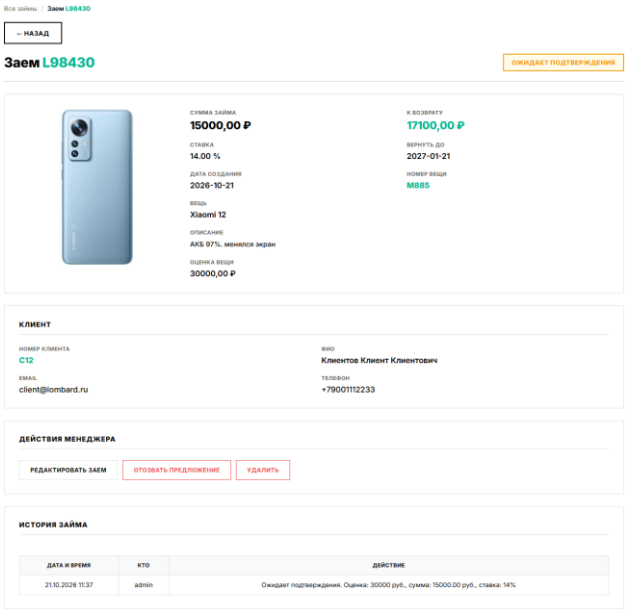
Таблица со всеми займами ломбарда - основной рабочий экран менеджера. Поиск работает по номеру займа, ФИО и номеру клиента, есть фильтр по статусу и сортировка по номеру, сумме, дате и статусу. В строке займа - кнопки просмотра, редактирования и удаления.



Скриншот 16 (Раздел «Все займы» для роли «Администратор»)

4.11 Раздел «Оценка и одобрение заявки»

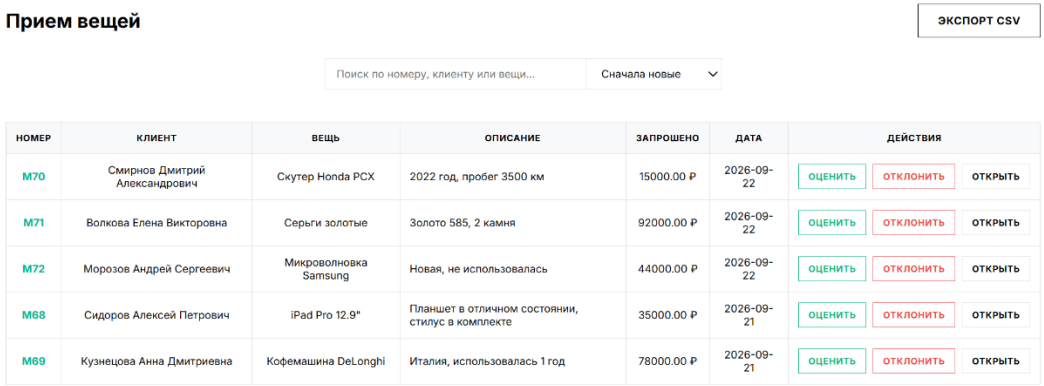
Менеджер открывает заявку, просматривает данные клиента, описание залога и фотографии, указывает оценку имущества, одобренную сумму и процентную ставку. После сохранения заявка переводится в статус «Ожидает подтверждения», клиенту отправляется уведомление с условиями.



Скриншот 17 (Форма оценки залога и одобрения займа)

4.12 Раздел «Прием вещей»

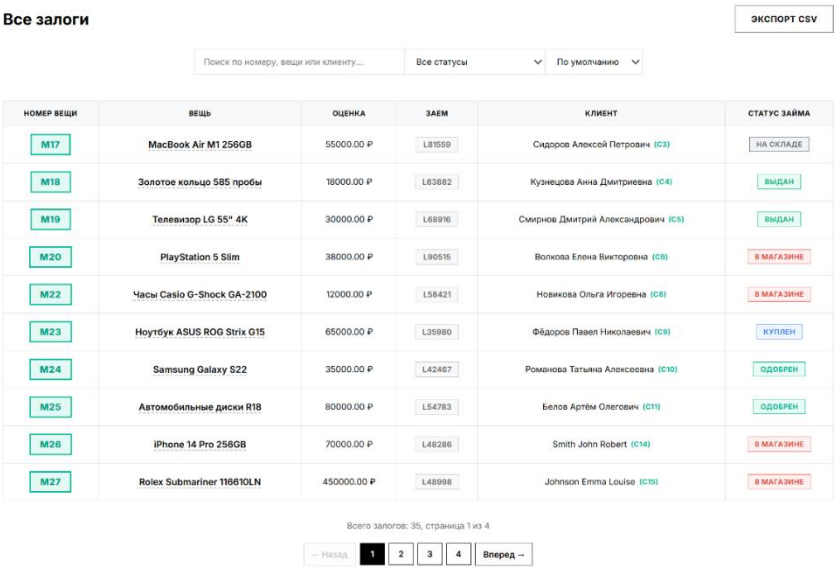
Это очередь новых заявок в статусе «В ожидании». Из нее менеджер переходит к оценке, правит описание залога или сразу отклоняет заявку.



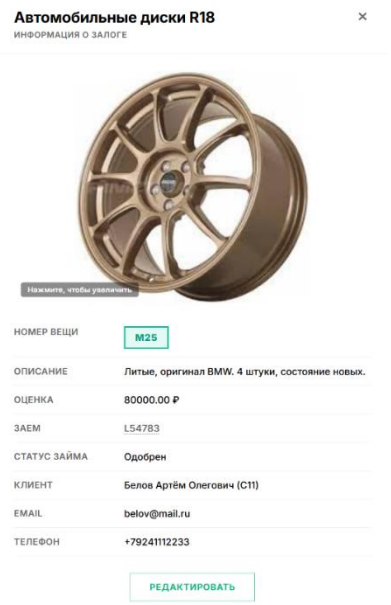
Скриншот 18 (Раздел «Прием вещей» для роли «Менеджер»)

4.13 Раздел «Залоги»

Список всех вещей, принятых в залог, с поиском, фильтром по статусу займа и сортировкой. Чтобы не уходить со страницы, подробности открываются в модальном окне по клику на название: фотографии, описание, оценка, связанный заем и клиент.



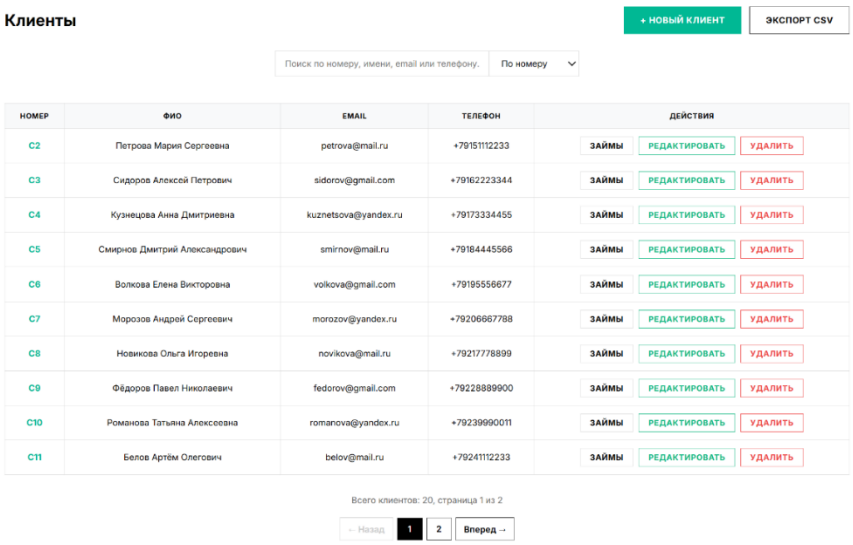
Скриншот 19 (Раздел «Залоги» для роли «Менеджер»)



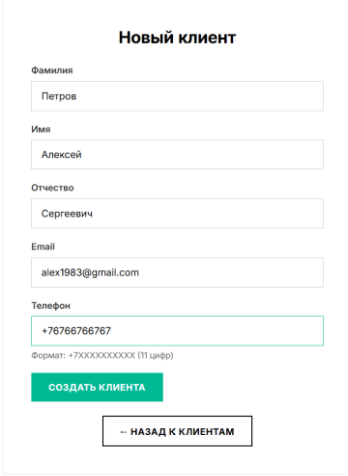
Скриншот 20 (Модальное окно с подробной информацией о залоге)

4.14 Раздел «Клиенты»

Раздел реализует полный CRUD по клиентам: создание, просмотр, редактирование и удаление (клиента с займами удалить нельзя - система выдаст сообщение). Поиск идет по номеру, ФИО, email и телефону, список можно выгрузить в CSV.



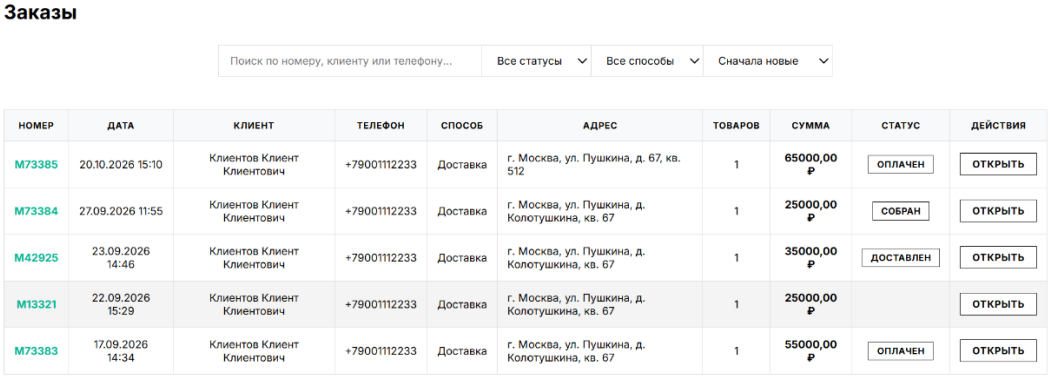
Скриншот 21 (Раздел «Клиенты» для роли «Администратор»)



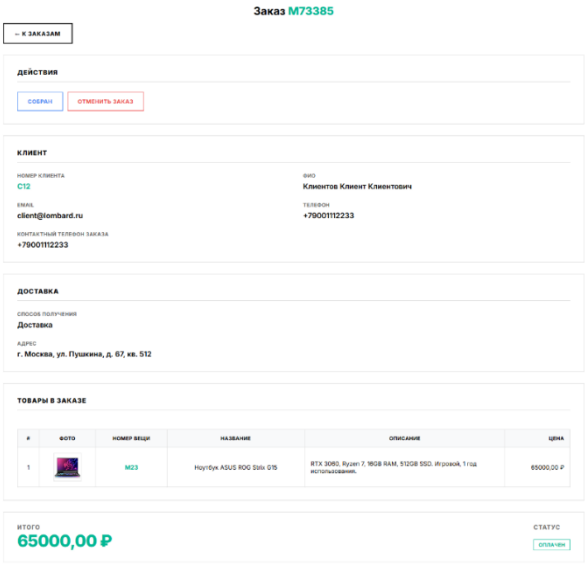
Скриншот 22 (Форма создания нового клиента)

4.15 Раздел «Заказы»

Все заказы магазина с поиском, фильтрами по статусу и способу получения и сортировкой. В карточке заказа сотрудник меняет статус (собран, доставлен, отменен), и клиент сразу получает уведомление.



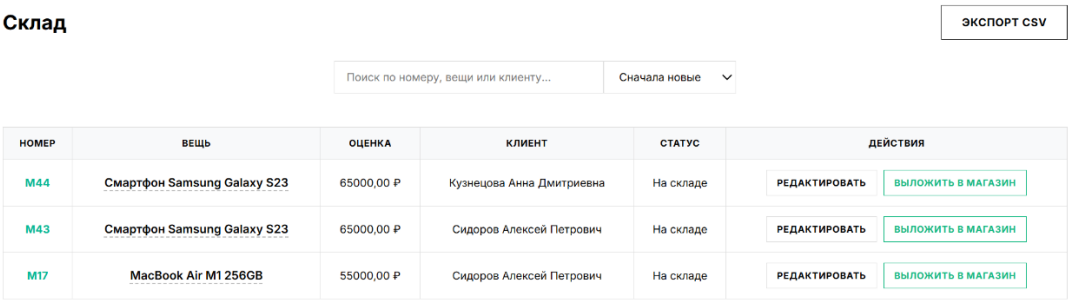
Скриншот 23 (Раздел «Заказы» для роли «Администратор»)



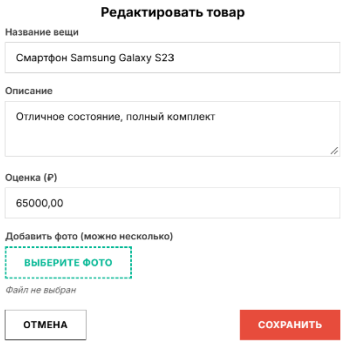
Скриншот 24 (Детали заказа с возможностью изменения статуса)

4.16 Раздел «Склад»

На склад попадают вещи по займам, просроченным более 30 дней. Перед публикацией в магазине сотрудник может поправить название, описание, оценку и фотографии; удалить товар может только администратор.



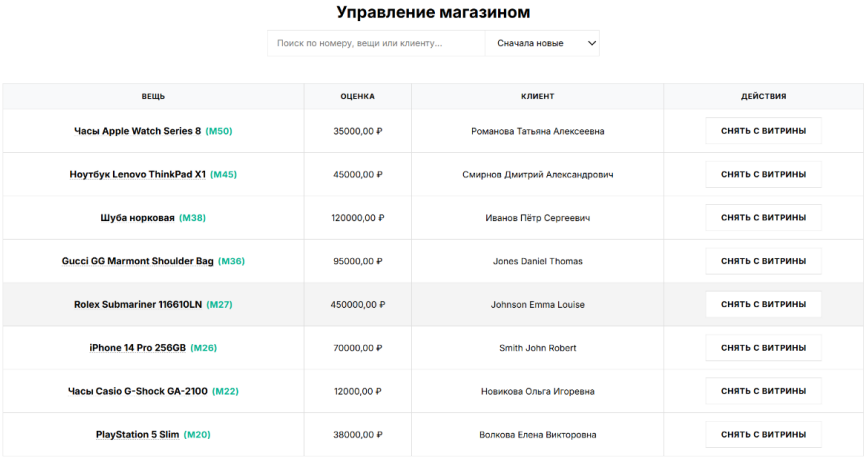
Скриншот 25 (Раздел «Склад» для роли «Менеджер»)



Скриншот 26 (Модальное окно редактирования товара на складе)

4.17 Раздел «Управление магазином»

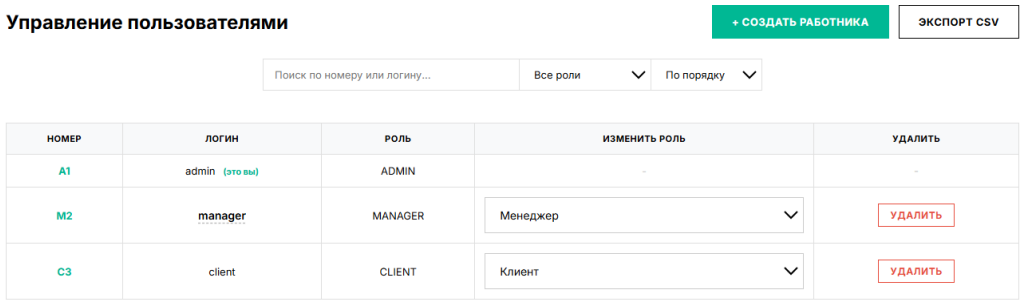
Список товаров на витрине. Любой товар можно снять с продажи - он вернется на склад.



Скриншот 27 (Раздел «Управление магазином» для роли «Менеджер»)

4.18 Раздел «Пользователи» (только администратор)

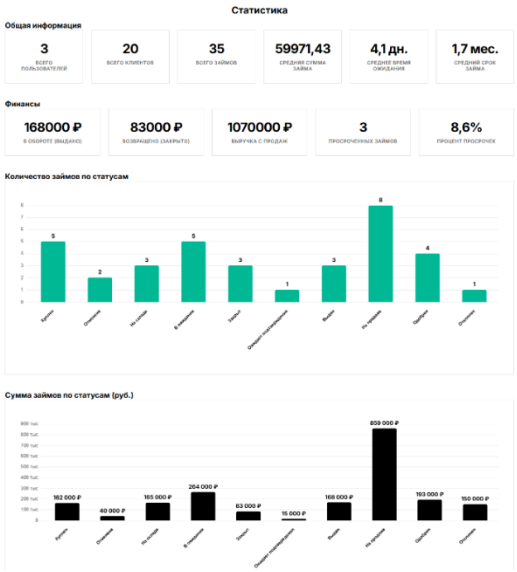
Здесь администратор создает сотрудников, меняет роли и удаляет учетные записи, а также выгружает и загружает пользователей через CSV. Свою учетную запись он изменить или удалить не может - так в системе не останется без администратора.



Скриншот 28 (Раздел «Пользователи» для роли «Администратор»)

4.19 Раздел «Статистика» (только администратор)

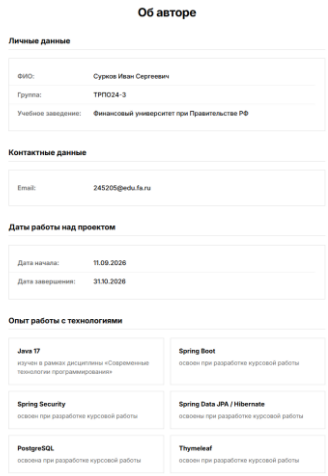
Отображаются общие показатели: количество пользователей, клиентов и займов, средняя сумма займа, среднее время ожидания решения, средний срок займа, финансовые показатели и доля просроченных займов. Две гистограммы (Chart.js) показывают количество и сумму займов по статусам.



Скриншот 29 (Раздел «Статистика» для роли «Администратор»)

4.20 Раздел «Об авторе»

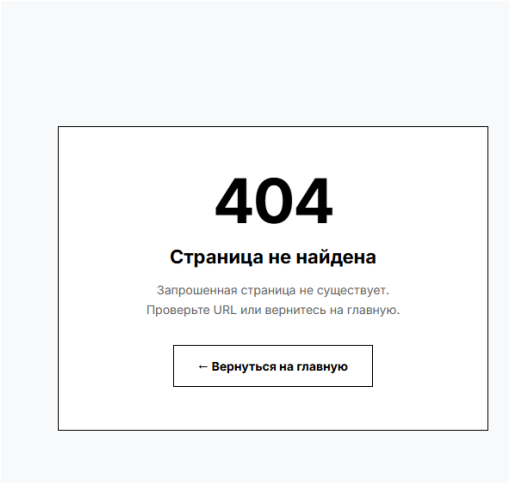
Отдельная страница (/about) содержит сведения об авторе курсовой работы: ФИО, группа, учебное заведение, контактные данные, описание опыта работы с использованными технологиями, даты начала и завершения работы над проектом.



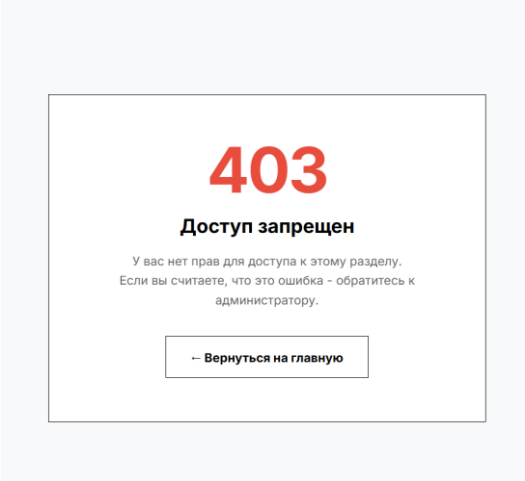
Скриншот 30 (Страница «Об авторе»)

4.21 Страницы ошибок

При возникновении ошибок пользователю отображаются отдельные страницы: 403 (доступ запрещен), 404 (страница не найдена), 500 (внутренняя ошибка сервера). Каждая страница содержит понятное описание проблемы и рекомендации (например, «вернитесь на главную»).



Скриншот 31 (Страница ошибки 404, страница не найдена)



Скриншот 32 (Страница ошибки 403 - Доступ запрещен)

5 СТРУКТУРА РЕАЛИЗОВАННЫХ КЛАССОВ И ИХ НАЗНАЧЕНИЕ

Классы проекта по пакетам и их назначение приведены в таблицах 1-6.

Таблица 1 - Пакет com.lombard.model (сущности предметной области):

Класс	Назначение
User	Пользователь системы: логин, пароль, роль, уникальный номер
Client	Клиент ломбарда: ФИО, телефон, email, номер, связь с пользователем
Loan	Заем: номер, сумма, ставка, статус, даты, связь с клиентом
Collateral	Залоговое имущество: номер, название, описание, оценка, фотографии
Order	Заказ товара: номер, сумма, статус, способ доставки, адрес

OrderItem	Позиция заказа: связь с заказом и займом, название, цена
Account	Лицевой счет клиента: баланс, связь с клиентом
AccountTransaction	Транзакция по счету: сумма, тип, описание, дата
CartItem	Элемент корзины: связь клиента и товара
Notification	Уведомление: тип события, параметры, статус прочтения
LoanHistory	История изменений займа: действие, автор, дата
NumberSequence	Последовательность для генерации уникальных номеров

Таблица 2 - Пакет com.lombard.repository (доступ к данным):

Класс	Назначение
UserRepository	Работа с пользователями: поиск по логину, по роли
ClientRepository	Работа с клиентами: поиск, проверка существования по телефону
LoanRepository	Работа с займами: поиск по клиенту, агрегатные запросы
CollateralRepository	Работа с залогами: поиск по любому полю
OrderRepository	Работа с заказами: поиск по клиенту
OrderItemRepository	Работа с позициями заказов
AccountRepository	Работа со счетами: поиск по клиенту, пессимистичная блокировка
AccountTransactionRepository	Работа с транзакциями
CartItemRepository	Работа с корзиной
NotificationRepository	Работа с уведомлениями
LoanHistoryRepository	Работа с историей займов
NumberSequenceRepository	Работа с последовательностями номеров

Таблица 3 - Пакет com.lombard.service (бизнес-логика):

Класс	Назначение
UserService	Регистрация пользователей, смена роли и пароля, удаление
ClientService	Создание, редактирование, удаление клиентов, поиск
LoanService	Создание займов, изменение статуса, продление, обработка просрочек
CollateralService	Работа с залогами, управление фотографиями
OrderService	Создание заказов, изменение статуса
CartService	Работа с корзиной
AccountService	Пополнение, вывод средств, покупка, история операций
NotificationService	Отправка уведомлений клиентам и сотрудникам
LoanHistoryService	Запись истории изменений займа
NumberSequenceService	Генерация уникальных номеров
FileStorageService	Сохранение и удаление загруженных фотографий
ManagerStatsService	Статистика действий менеджеров
OverdueLoanScheduler	Ночная проверка просрочек по расписанию (@Scheduled)

Таблица 4 - Пакет com.lombard.controller (обработка HTTP-запросов):

Класс	Назначение
AuthController	Вход, регистрация, страница 403
HomeController	Главная страница
ClientController	Функции клиента: займы, профиль
LoanController	Управление займами (для клиентов и сотрудников)
CollateralController	Управление залогами
AdminClientController	Управление клиентами (сотрудники)
AdminController	Управление пользователями и статистика
AdminManagerController	Просмотр профилей менеджеров
AdminOrderController	Управление заказами
IntakeController	Прием вещей
WarehouseController	Склад
ShopController	Магазин
CartController	Корзина и оформление заказа
OrderController	Заказы клиента
AccountController	Лицевой счет клиента
NotificationController	Уведомления
ProfileController	Смена пароля
AboutController	Страница «Об авторе»
SupportController	Страница поддержки
GlobalExceptionHandler	Глобальная обработка исключений

Таблица 5 - Пакет com.lombard.config (конфигурация):

Класс	Назначение
SecurityConfig	Настройка Spring Security: правила доступа, CSRF, аутентификация
WebConfig	Настройка обработки статических ресурсов (фотографий)
DataInitializer	Инициализация последовательностей номеров при запуске
GlobalModelAttributes	Добавление в модель всех страниц общих данных: счетчики корзины, уведомлений и активных займов (клиент), количество заявок на обработку (сотрудники)
SchedulingConfig	Включение задач по расписанию

Таблица 6 - Пакет com.lombard.util (утилиты):

Класс	Назначение
CsvExporter	Формирование CSV-файлов с защитой от формул Excel
TransactionUtils	Откат текущей транзакции, когда ошибка обработана в блоке catch

ЗАКЛЮЧЕНИЕ

В результате работы создано веб-приложение ломбарда на Java, Spring Boot, Hibernate, PostgreSQL и Thymeleaf с тремя ролями: клиент, менеджер и администратор. В нем реализован полный путь займа - от заявки клиента и оценки вещи менеджером до погашения либо, при просрочке, перевода вещи на склад и продажи через магазин. Для клиентов, займов, залогов, товаров и пользователей реализованы операции создания, просмотра, редактирования и удаления, поиск и сортировка, для администратора - статистика с диаграммами.

Больше всего времени ушло не на экраны, а на корректность операций с деньгами и статусами. Пришлось ввести блокировку строки счета и счетчика номеров, явный откат транзакции при обработанной ошибке и карту допустимых переходов статусов займа. Часть ошибок нашли автотесты: 459 тестов при покрытии кода 83 % по данным JaCoCo, сборка не проходит, если покрытие опускается ниже 80 %.

Ограничения у системы тоже есть: пополнение и вывод средств только имитируются, уведомления приходят лишь внутри приложения, а поиск по займам выполняется в памяти и при большом объеме данных его стоит перенести в запросы к базе. Эти места - первые кандидаты на доработку вместе с подключением платежного сервиса и отправкой уведомлений по email.

Поставленная цель достигнута: все задачи из введения выполнены, а приложение отвечает требованиям к курсовой работе по дисциплине «Современные технологии программирования».

СПИСОК ЛИТЕРАТУРЫ

1. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. - СПб.: Питер, 2019. - 368 с.

2. Уоллс К. Spring в действии. - М.: ДМК Пресс, 2018. - 520 с.

3. Фаулер М. Архитектура корпоративных программных приложений. - М.: Вильямс, 2020. - 544 с.

4. Эккель Б. Философия Java. - СПб.: Питер, 2019. - 1168 с.

5. Документация Chart.js [Электронный ресурс]. - URL: https://www.chartjs.org/docs/ (дата обращения: 20.10.2026).

6. Документация Hibernate ORM [Электронный ресурс]. - URL: https://hibernate.org/orm/documentation/ (дата обращения: 20.10.2026).

7. Документация PostgreSQL [Электронный ресурс]. - URL: https://www.postgresql.org/docs/ (дата обращения: 20.10.2026).

8. Документация Spring Boot [Электронный ресурс]. - URL: https://spring.io/projects/spring-boot (дата обращения: 20.10.2026).

9. Документация Spring Data JPA [Электронный ресурс]. - URL: https://spring.io/projects/spring-data-jpa (дата обращения: 20.10.2026).

10. Документация Spring Security [Электронный ресурс]. - URL: https://spring.io/projects/spring-security (дата обращения: 20.10.2026).

11. Документация Thymeleaf [Электронный ресурс]. - URL: https://www.thymeleaf.org/documentation.html (дата обращения: 20.10.2026).

12. Официальный сайт Java [Электронный ресурс]. - URL: https://www.java.com/ (дата обращения: 20.10.2026).

13. Официальный сайт Maven [Электронный ресурс]. - URL: https://maven.apache.org/ (дата обращения: 20.10.2026).

14. Руководство по HTML5 и CSS3 [Электронный ресурс]. - URL: https://developer.mozilla.org/ru/ (дата обращения: 20.10.2026).

ПРИЛОЖЕНИЯ

В приложениях приведен исходный код ключевых классов программы: конфигурации безопасности, контроллера и сервиса займов, глобального обработчика исключений и основных полей модели Loan (листинги приведены без строк import), а также файл настроек application.properties. Полный исходный код всего проекта представлен в архиве, прилагаемом к курсовой работе, а также в репозитории GitHub: https://github.com/Ivanpython2006/lombard-project.

ПРИЛОЖЕНИЕ А

Класс SecurityConfig

```
package com.lombard.config;
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    private final UserRepository userRepository;
    public SecurityConfig(UserRepository userRepository) {
        this.userRepository = userRepository;
    }
    @Bean
    // храню CSRF-токен в куках (удобнее с формами
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        CookieCsrfTokenRepository csrfRepo =
CookieCsrfTokenRepository.withHttpOnlyFalse();
        csrfRepo.setCookiePath("/");
        csrfRepo.setCookieName("XSRF-TOKEN");
        csrfRepo.setHeaderName("X-XSRF-TOKEN");
        CsrfTokenRequestAttributeHandler csrfHandler = new
CsrfTokenRequestAttributeHandler();
        csrfHandler.setCsrfRequestAttributeName(null);
        http
                .csrf(csrf -> csrf
                        .csrfTokenRepository(csrfRepo)
                        .csrfTokenRequestHandler(csrfHandler)
                )
                .sessionManagement(session -> session
                        // сессия по необходимости
                        .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
                )
                .authorizeHttpRequests(auth -> auth
                        .requestMatchers( // пускаю всех на логин регистрацию и к статике
                                "/login", "/register",
                                "/style.css", "/*.css", "/*.js", "/*.png", "/*.jpg",
"/*.ico",
                                "/css/**", "/js/**", "/images/**",
                                "/favicon.ico"
                        ).permitAll()
                        // фото залогов показываю только вошедшим пользователям
                        .requestMatchers("/uploads/**").authenticated()
                        // страницы ошибок тоже открыты
                        .requestMatchers("/403", "/404", "/error",
"/error/**").permitAll()
                        // разделы для админов и менеджеров
                        .requestMatchers("/admin/notifications",
"/admin/notifications/**").hasAnyRole("ADMIN", "MANAGER")
```

```
                        .requestMatchers("/admin/warehouse",
"/admin/warehouse/**").hasAnyRole("ADMIN", "MANAGER")
                        .requestMatchers("/admin/orders",
"/admin/orders/**").hasAnyRole("ADMIN", "MANAGER")
                        .requestMatchers("/admin/clients",
"/admin/clients/**").hasAnyRole("ADMIN", "MANAGER")
                        .requestMatchers("/admin/intake",
"/admin/intake/**").hasAnyRole("ADMIN", "MANAGER")
                        //а это только для самого адмна
                        .requestMatchers("/admin/stats",
"/admin/stats/**").hasRole("ADMIN")
                        .requestMatchers("/admin/users",
"/admin/users/**").hasRole("ADMIN")
                        .requestMatchers("/admin/managers",
"/admin/managers/**").hasRole("ADMIN")
                        .requestMatchers("/admin/**").hasRole("ADMIN")
                        // займы идут менеджерам и админам клиентам только просмотр своих
                        .requestMatchers("/loans/export").hasAnyRole("ADMIN", "MANAGER")
                        .requestMatchers("/loans/{id}/approve").hasAnyRole("ADMIN",
"MANAGER")
                        .requestMatchers("/loans/{id}/status").hasAnyRole("ADMIN",
"MANAGER")
                        .requestMatchers("/loans/{id}/edit").hasAnyRole("ADMIN",
"MANAGER")
                        .requestMatchers("/loans/{id}/extend").hasAnyRole("ADMIN",
"MANAGER")
                        .requestMatchers("/loans/{id}/delete").hasAnyRole("ADMIN",
"MANAGER")
                        .requestMatchers("/loans", "/loans/").hasAnyRole("ADMIN",
"MANAGER")
                        .requestMatchers("/loans/**").authenticated()
                        // залоги только сотрудники
                        .requestMatchers("/collaterals/**").hasAnyRole("ADMIN",
"MANAGER")
                        // поиск создание редактирование и удаление клиентов только
сотрудникам
                        .requestMatchers("/client/search", "/client/new",
"/client/*/edit", "/client/export", "/client/*/delete")
                        .hasAnyRole("ADMIN", "MANAGER")
                        .requestMatchers("/client/loans",
"/client/loans/new").hasRole("CLIENT")
                        .requestMatchers("/client/**").hasAnyRole("ADMIN", "MANAGER",
"CLIENT")
                        .requestMatchers("/support").hasRole("CLIENT") //оставшиеся
                        .requestMatchers("/profile/**").authenticated()
                        .requestMatchers("/shop", "/shop/**").authenticated()
                        .requestMatchers("/about").authenticated()
                        .anyRequest().authenticated()
                )
                .exceptionHandling(ex -> ex // если нет прав кидаю на 403
                        .accessDeniedPage("/403")
                )
                .formLogin(form -> form
                        .loginPage("/login")
                        .defaultSuccessUrl("/", true)
                        .permitAll()
                )
                .logout(logout -> logout
                        .logoutUrl("/logout")
                        .logoutSuccessUrl("/login?logout")
                        .permitAll()
                );
        return http.build();
    }
    @Bean
    public UserDetailsService userDetailsService() {
        return username -> {
            Optional<User> user = userRepository.findByUsername(username);
            if (user.isPresent()) {
                User u = user.get();
                return org.springframework.security.core.userdetails.User
```

```
                        .withUsername(u.getUsername())
                        .password(u.getPassword())
                        .roles(u.getRole())
                        .build();
            } else {
                throw new UsernameNotFoundException("Пользователь не найден");
            }
        };
    }
    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

ПРИЛОЖЕНИЕ Б

Класс LoanController

```
package com.lombard.controller;
@Controller
@RequestMapping("/loans")
public class LoanController {
    private static final int PAGE_SIZE = 10;
    private static final Map<String, Set<String>> ALLOWED_TRANSITIONS = Map.ofEntries(
            Map.entry("В ожидании", Set.of("Одобрен", "Отклонен", "Ожидает
подтверждения", "Отменена")),
            Map.entry("Одобрен", Set.of("Выдан", "Отклонен", "Отменена")),
            Map.entry("Ожидает подтверждения", Set.of("Выдан", "Отклонен", "Отменена")),
            Map.entry("Выдан", Set.of("Закрыт", "Просрочен", "На продаже", "На складе")),
            Map.entry("Просрочен", Set.of("Закрыт", "На складе", "На продаже")),
            Map.entry("На складе", Set.of("На продаже", "Закрыт")),
            Map.entry("На продаже", Set.of("Куплен", "На складе")),
            Map.entry("Закрыт", Set.of()),
            Map.entry("Отклонен", Set.of()),
            Map.entry("Отменена", Set.of()),
            Map.entry("Куплен", Set.of())
    );
    private final LoanService loanService;
    private final ClientService clientService;
    private final UserService userService;
    private final LoanHistoryService loanHistoryService;
    private final NotificationService notificationService;
    private final CollateralService collateralService;
    public LoanController(LoanService loanService,
                          ClientService clientService,
                          UserService userService,
                          LoanHistoryService loanHistoryService,
                          NotificationService notificationService,
                          CollateralService collateralService) {
        this.loanService = loanService;
        this.clientService = clientService;
        this.userService = userService;
        this.loanHistoryService = loanHistoryService;
        this.notificationService = notificationService;
        this.collateralService = collateralService;
    }
    private boolean isAdminOrManager(Authentication auth) {
        return auth.getAuthorities().stream()
                .anyMatch(a -> a.getAuthority().equals("ROLE_ADMIN")
                        || a.getAuthority().equals("ROLE_MANAGER"));
    }
    private BigDecimal parseDecimal(String value) {
        if (value == null || value.isBlank()) return BigDecimal.ZERO;
        try {
            String cleaned = value.replaceAll("[^0-9.-]", "").trim();
            if (cleaned.isEmpty()) return BigDecimal.ZERO;
            return new BigDecimal(cleaned);
```

```
        } catch (NumberFormatException e) {
            return BigDecimal.ZERO;
        }
    }
    @GetMapping
    public String listLoans(@RequestParam(defaultValue = "id") String sort,
                            @RequestParam(required = false) String status,
                            @RequestParam(required = false) String clientName,
                            @RequestParam(defaultValue = "1") int page,
                            Model model, Authentication auth) {
        loanService.processOverdueLoans(notificationService);
        if (isAdminOrManager(auth)) {
            String statusFilter = (status == null || status.isEmpty()) ? null : status;
            List<Loan> loans = new ArrayList<>(loanService.searchLoans(statusFilter,
clientName, null));
            switch (sort) {
                case "date_asc":
                    loans.sort(Comparator.comparing(Loan::getCreatedDate,
Comparator.nullsLast(Comparator.naturalOrder())));
                    break;
                case "amount_desc":
                    loans.sort(Comparator.comparing(Loan::getAmount,
Comparator.nullsLast(Comparator.reverseOrder())));
                    break;
                case "amount_asc":
                    loans.sort(Comparator.comparing(Loan::getAmount,
Comparator.nullsLast(Comparator.naturalOrder())));
                    break;
                case "status":
                    loans.sort(Comparator.comparing(Loan::getStatus,
Comparator.nullsLast(Comparator.naturalOrder())));
                    break;
                case "id":
                default:
                    loans.sort(Comparator.comparing(Loan::getId).reversed());
                    break;
            }
            int totalItems = loans.size();
            int totalPages = (int) Math.ceil((double) totalItems / PAGE_SIZE);
            if (totalPages == 0) totalPages = 1;
            if (page < 1) page = 1;
            if (page > totalPages) page = totalPages;
            int fromIndex = (page - 1) * PAGE_SIZE;
            int toIndex = Math.min(fromIndex + PAGE_SIZE, totalItems);
            List<Loan> pageContent = totalItems > 0 ? loans.subList(fromIndex, toIndex) :
List.of();
            model.addAttribute("loans", pageContent);
            model.addAttribute("currentPage", page);
            model.addAttribute("totalPages", totalPages);
            model.addAttribute("totalItems", totalItems);
            model.addAttribute("isAdminOrManager", true);
        } else {
            User user = userService.findByUsername(auth.getName());
            Client client = (user == null) ? null :
clientService.findByUserId(user.getId());
            model.addAttribute("loans", client != null
                    ? loanService.findByClientId(client.getId())
                    : List.of());
            model.addAttribute("isAdminOrManager", false);
        }
        model.addAttribute("sort", sort);
        model.addAttribute("status", status);
        model.addAttribute("clientName", clientName);
        return "loans";
    }
    @GetMapping("/export")
    public void exportLoans(HttpServletResponse response,
                            Authentication auth) throws IOException {
        if (!isAdminOrManager(auth)) {
            response.sendRedirect("/403");
            return;
```

```
        }
        List<Loan> loans = loanService.findAll();
        List<String[]> rows = new ArrayList<>();
        for (Loan l : loans) {
            String collateralName = "";
            String collateralPrice = "";
            if (!l.getCollaterals().isEmpty()) {
                Collateral c = l.getCollaterals().get(0);
                collateralName = c.getName() != null ? c.getName() : "";
                collateralPrice = c.getEstimatedPrice() != null
                        ? c.getEstimatedPrice().toPlainString() : "";
            }
            rows.add(new String[]{
                    String.valueOf(l.getId()),
                    l.getClient() != null ? CsvExporter.esc(l.getClient().getFullName())
: "",
                    CsvExporter.esc(collateralName),
                    collateralPrice,
                    l.getAmount() != null ? l.getAmount().toPlainString() : "",
                    l.getRate() != null ? l.getRate().toPlainString() : "",
                    l.getRate() != null ? l.getTotalToReturn().toPlainString() : "",
                    l.getStatus() != null ? CsvExporter.esc(l.getStatus()) : "",
                    l.getCreatedDate() != null ? l.getCreatedDate().toString() : "",
                    l.getDueDate() != null ? l.getDueDate().toString() : ""
            });
        }
        CsvExporter.write(response, "loans.csv", rows,
                "ID", "Клиент", "Вещь", "Оценка", "Сумма",
                "Ставка %", "К возврату", "Статус", "Создан", "Вернуть до");
    }
    @GetMapping("/{id}")
    public String loanDetails(@PathVariable Long id, Model model, Authentication auth,
                              RedirectAttributes ra) {
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        // ИСПРАВЛЕНИЕ: Чтобы Thymeleaf не падал, если photoUrls = null
        if (loan.getCollaterals() != null) {
            loan.getCollaterals().forEach(c -> {
                if (c.getPhotoUrls() == null) c.setPhotoUrls(new ArrayList<>());
            });
        }
        boolean isAdminOrManager = isAdminOrManager(auth);
        if (!isAdminOrManager) {
            User user = userService.findByUsername(auth.getName());
            Client client = (user == null) ? null :
clientService.findByUserId(user.getId());
            if (loan.getClient() == null || client == null
                    || !loan.getClient().getId().equals(client.getId())) {
                ra.addFlashAttribute("error", "Заем не найден");
                return "redirect:/loans";
            }
        }
        model.addAttribute("loan", loan);
        model.addAttribute("isAdminOrManager", isAdminOrManager);
        model.addAttribute("history", loanHistoryService.findByLoanId(id));
        return "loan_details";
    }
    @GetMapping("/{id}/approve")
    public String approveForm(@PathVariable Long id, Model model, Authentication auth,
                              RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        model.addAttribute("loan", loan);
        return "loan_approve";
```

```
    }
    @PostMapping("/{id}/approve")
    @Transactional
    public String approveLoan(@PathVariable Long id,
                              @RequestParam(defaultValue = "0") String rate,
                              @RequestParam(defaultValue = "0") String estimatedPrice,
                              @RequestParam(defaultValue = "0") String approvedAmount,
                              @RequestParam(required = false) String collateralName,
                              @RequestParam(required = false) String
collateralDescription,
                              Authentication auth,
                              RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        BigDecimal rateVal = parseDecimal(rate);
        BigDecimal estimatedPriceVal = parseDecimal(estimatedPrice);
        BigDecimal approvedAmountVal = parseDecimal(approvedAmount);
        if (estimatedPriceVal.compareTo(BigDecimal.ZERO) <= 0) {
            ra.addFlashAttribute("error", "Укажите корректную оценку вещи (больше 0)");
            return "redirect:/loans/" + id + "/approve";
        }
        if (rateVal.compareTo(BigDecimal.ZERO) < 0 ||
rateVal.compareTo(BigDecimal.valueOf(100)) > 0) {
            ra.addFlashAttribute("error", "Ставка должна быть в диапазоне 0-100%");
            return "redirect:/loans/" + id + "/approve";
        }
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        BigDecimal safeAmount = (approvedAmountVal != null &&
approvedAmountVal.compareTo(BigDecimal.ZERO) > 0)
                ? approvedAmountVal : loan.getAmount();
        loan.setRate(rateVal);
        loan.setAmount(safeAmount);
        loan.setStatus("Ожидает подтверждения");
        loan.setAgreedAmount(safeAmount);
        loan.setClientApproved(null);
        loan.setDecisionDate(LocalDate.now());
        loanService.save(loan);
        StringBuilder changes = new StringBuilder();
        if (!loan.getCollaterals().isEmpty()) {
            Collateral c = loan.getCollaterals().get(0);
            String oldName = c.getName();
            String oldDesc = c.getDescription();
            c.setEstimatedPrice(estimatedPriceVal);
            if (collateralName != null && !collateralName.isBlank()) {
                String newName = collateralName.trim();
                if (oldName == null || !oldName.equals(newName)) {
                    changes.append("Название: «").append(oldName != null ? oldName : "-")
                            .append("» -> «").append(newName).append("». ");
                    c.setName(newName);
                }
            }
            if (collateralDescription != null) {
                String newDesc = collateralDescription.trim();
                String safeOld = oldDesc != null ? oldDesc : "";
                if (!safeOld.equals(newDesc)) {
                    changes.append("Описание: «").append(safeOld.isEmpty() ? "-" :
safeOld)
                            .append("» -> «").append(newDesc.isEmpty() ? "-" :
newDesc).append("». ");
                    c.setDescription(newDesc);
                }
            }
        }
        loanService.save(loan);
        String itemName = loanService.getItemName(loan);
        BigDecimal totalToReturn = safeAmount.add(
                safeAmount.multiply(rateVal).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP)
```

```
        ).setScale(2, RoundingMode.HALF_UP);
        loanHistoryService.log(loan,
                "Ожидает подтверждения. Оценка: " + estimatedPriceVal.toPlainString() +
                        " руб., сумма: " + safeAmount.toPlainString() +
                        " руб., ставка: " + rateVal.toPlainString() + "%",
                auth.getName());
        if (loan.getClient() != null) {
            notificationService.notify(loan.getClient(), "LOAN_APPROVED", "WARNING",
                    itemName,
                    loan.getAmount() != null ? loan.getAmount().toPlainString() : "0",
                    safeAmount.toPlainString(),
                    rateVal.toPlainString(),
                    totalToReturn.toPlainString()
            );
            if (changes.length() > 0) {
                notificationService.notify(loan.getClient(), "LOAN_CHANGED", "WARNING",
                        loan.getLoanNumber(),
                        changes.toString().trim()
                );
            }
        }
        ra.addFlashAttribute("success", "Заявка одобрена и отправлена клиенту на
подтверждение");
        return "redirect:/loans/" + id;
    }
    @GetMapping("/{id}/edit")
    public String editForm(@PathVariable Long id, Model model, Authentication auth,
                           RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        model.addAttribute("loan", loan);
        return "edit_loan";
    }
    @PostMapping("/{id}/edit")
    public String editSave(@PathVariable Long id,
                           @RequestParam(required = false) BigDecimal amount,
                           @RequestParam(required = false) BigDecimal rate,
                           @RequestParam(required = false) String status,
                           @RequestParam(required = false) String dueDate,
                           @RequestParam(value = "photos", required = false)
MultipartFile[] photos,
                           Authentication auth,
                           RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        if (amount != null && amount.compareTo(BigDecimal.ZERO) > 0)
loan.setAmount(amount);
        if (rate != null && rate.compareTo(BigDecimal.ZERO) >= 0) loan.setRate(rate);
        if (status != null && !status.isBlank()) loan.setStatus(status);
        if (dueDate != null && !dueDate.isBlank()) {
            try {
                loan.setDueDate(LocalDate.parse(dueDate));
            } catch (Exception ignored) {}
        }
        loanService.save(loan);
        if (photos != null && photos.length > 0
                && loan.getCollaterals() != null
                && !loan.getCollaterals().isEmpty()) {
            try {
                collateralService.appendPhotos(loan.getCollaterals().get(0), photos,
                        "item_" + loan.getCollaterals().get(0).getId());
            } catch (RuntimeException e) {
                ra.addFlashAttribute("error", "Ошибка загрузки фото: " + e.getMessage());
```

```
                return "redirect:/loans/" + id + "/edit";
            }
        }
        String logAmount = (amount != null) ? amount.toPlainString() + " руб." : "не
указана";
        String logRate = (rate != null) ? rate.toPlainString() + " %" : "не указана";
        String logStatus = (status != null && !status.isEmpty()) ? status : "без
изменений";
        loanHistoryService.log(loan,
                "Отредактирован менеджером: сумма=" + logAmount +
                        ", ставка=" + logRate +
                        ", статус=" + logStatus,
                auth.getName());
        ra.addFlashAttribute("success", "Заем обновлен");
        return "redirect:/loans/" + id;
    }
    @PostMapping("/{id}/status")
    public String changeStatus(@PathVariable Long id,
                               @RequestParam String status,
                               Authentication auth,
                               RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        String oldStatus = loan.getStatus();
        Set<String> allowed = ALLOWED_TRANSITIONS.getOrDefault(oldStatus, Set.of());
        if (!allowed.contains(status)) {
            ra.addFlashAttribute("error",
                    "Недопустимый переход: <" + oldStatus + "> -> <" + status + ">");
            return "redirect:/loans/" + id;
        }
        loan.setStatus(status);
        loanService.save(loan);
        String itemName = loanService.getItemName(loan);
        loanHistoryService.log(loan,
                "Статус: «" + oldStatus + "» -> «" + status + "»",
                auth.getName());
        if (loan.getClient() != null) {
            switch (status) {
                case "Отклонен":
                    notificationService.notify(loan.getClient(), "LOAN_REJECTED",
"DANGER",
                            loan.getLoanNumber(), itemName);
                    break;
                case "Закрыт":
                    notificationService.notify(loan.getClient(), "LOAN_CLOSED",
"SUCCESS",
                            loan.getLoanNumber(), itemName);
                    break;
                case "Выдан":
                    notificationService.notify(loan.getClient(), "LOAN_ISSUED",
"SUCCESS",
                            loan.getLoanNumber(),
                            itemName,
                            loan.getTotalToReturn().toPlainString(),
                            String.valueOf(loan.getDueDate()));
                    break;
                case "На продаже":
                    notificationService.notify(loan.getClient(), "LOAN_ON_SALE",
"WARNING",
                            loan.getLoanNumber(), itemName);
                    break;
                default:
                    notificationService.notify(loan.getClient(), "LOAN_STATUS", "INFO",
                            loan.getLoanNumber(), itemName, status);
                    break;
            }
        }
```

```
        ra.addFlashAttribute("success", "Статус изменен на «" + status + "»");
        return "redirect:/loans/" + id;
    }
    @PostMapping("/{id}/withdraw")
    public String withdrawOffer(@PathVariable Long id,
                                Authentication auth,
                                RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        if (!"Ожидает подтверждения".equals(loan.getStatus())) {
            ra.addFlashAttribute("error", "Предложение нельзя отозвать в текущем
статусе");
            return "redirect:/loans/" + id;
        }
        loan.setStatus("Отклонен");
        loan.setClientApproved(false);
        loanService.save(loan);
        String itemName = loanService.getItemName(loan);
        loanHistoryService.log(loan,
                "Менеджер отозвал предложение по заявке",
                auth.getName());
        if (loan.getClient() != null) {
            notificationService.notify(loan.getClient(), "LOAN_OFFER_WITHDRAWN",
"DANGER",
                    loan.getLoanNumber(), itemName);
        }
        ra.addFlashAttribute("success", "Предложение отозвано. Заявка отклонена");
        return "redirect:/loans/" + id;
    }
    @PostMapping("/{id}/extend")
    public String extendLoan(@PathVariable Long id,
                             @RequestParam(defaultValue = "1") int months,
                             Authentication auth,
                             RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        Loan loan = loanService.findById(id);
        if (loan == null) {
            ra.addFlashAttribute("error", "Заем не найден");
            return "redirect:/loans";
        }
        loanService.extendLoan(id, months);
        Loan updated = loanService.findById(id);
        String itemName = loanService.getItemName(updated);
        loanHistoryService.log(updated,
                "Продлен на " + months + " мес. Новая дата возврата: " +
updated.getDueDate(),
                auth.getName());
        if (updated.getClient() != null) {
            notificationService.notify(updated.getClient(), "LOAN_EXTENDED", "INFO",
                    updated.getLoanNumber(),
                    itemName,
                    String.valueOf(months),
                    String.valueOf(updated.getDueDate()));
        }
        ra.addFlashAttribute("success", "Заем продлен на " + months + " мес.");
        return "redirect:/loans/" + id;
    }
    @PostMapping("/{id}/delete")
    public String deleteLoan(@PathVariable Long id,
                             @RequestParam(required = false) Integer page,
                             @RequestParam(required = false) String sort,
                             @RequestParam(required = false) String status,
                             @RequestParam(required = false) String clientName,
                             Authentication auth,
                             RedirectAttributes ra) {
        if (!isAdminOrManager(auth)) return "redirect:/loans";
        loanService.delete(id);
```

```
        ra.addFlashAttribute("success", "Заявка удалена");
        ra.addAttribute("page", page != null ? page : 1);
        if (sort != null && !sort.isEmpty()) ra.addAttribute("sort", sort);
        if (status != null && !status.isEmpty()) ra.addAttribute("status", status);
        if (clientName != null && !clientName.isEmpty()) ra.addAttribute("clientName",
clientName);
        return "redirect:/loans";
    }
}
```

ПРИЛОЖЕНИЕ В

Класс LoanService

```
package com.lombard.service;
@Service
public class LoanService {
    // через сколько дней просрочки перевод залога на склад
    private static final int OVERDUE_TO_WAREHOUSE_DAYS = 30;
    private final LoanRepository loanRepository;
    private final NumberSequenceService numberSequenceService;
    private final CartItemRepository cartItemRepository;
    private final OrderItemRepository orderItemRepository;
    private final LoanHistoryRepository loanHistoryRepository;
    private final CollateralRepository collateralRepository;
    private final FileStorageService fileStorageService;
    private final LoanHistoryService loanHistoryService;
    public LoanService(LoanRepository loanRepository,
                       NumberSequenceService numberSequenceService,
                       CartItemRepository cartItemRepository,
                       OrderItemRepository orderItemRepository,
                       LoanHistoryRepository loanHistoryRepository,
                       CollateralRepository collateralRepository,
                       FileStorageService fileStorageService,
                       LoanHistoryService loanHistoryService) {
        this.loanRepository = loanRepository;
        this.numberSequenceService = numberSequenceService;
        this.cartItemRepository = cartItemRepository;
        this.orderItemRepository = orderItemRepository;
        this.loanHistoryRepository = loanHistoryRepository;
        this.collateralRepository = collateralRepository;
        this.fileStorageService = fileStorageService;
        this.loanHistoryService = loanHistoryService;
    }
    public List<Loan> findAll() { return loanRepository.findAll(); }
    public List<Loan> getAllLoans() { return loanRepository.findAll(); }
    public Page<Loan> findPaginated(int pageNo, int pageSize, String sort) {
        Sort s;
        switch (sort) {
            case "amount": s = Sort.by("amount"); break;
            case "date":   s = Sort.by("createdDate"); break;
            case "status": s = Sort.by("status"); break;
            default:       s = Sort.by("id"); break;
        }
        Pageable pageable = PageRequest.of(pageNo - 1, pageSize, s);
        return loanRepository.findAll(pageable);
    }
    public List<Loan> findAllSorted(String sort) {
        switch (sort) {
            case "amount": return loanRepository.findAll(Sort.by("amount"));
            case "date":   return loanRepository.findAll(Sort.by("createdDate"));
            case "status": return loanRepository.findAll(Sort.by("status"));
            default:       return loanRepository.findAll(Sort.by("id"));
        }
    }
    @Transactional(readOnly = true)
    public Loan findById(Long id) { return loanRepository.findById(id).orElse(null); }
    @Transactional
```

```
    public void save(Loan loan) {
        // если у займа еще нет номера то генерирую через sequence
        if (loan.getLoanNumber() == null || loan.getLoanNumber().isBlank()) {
            loan.setLoanNumber(numberSequenceService.nextNumber("L", "L"));
        }
        loanRepository.save(loan);
    }
    @Transactional
    public void delete(Long id) {
        Loan loan = loanRepository.findById(id).orElse(null);
        if (loan == null) return;
        cartItemRepository.deleteByLoanId(id); // чищу связи перед удалением
        List<OrderItem> orderItems = orderItemRepository.findByLoanId(id);
        for (OrderItem oi : orderItems) {
            oi.setLoan(null);
        }
        orderItemRepository.saveAll(orderItems);
        loanHistoryRepository.deleteByLoanId(id);
        // удаляю фотки с диска
        for (Collateral c : loan.getCollaterals()) {
            if (c.getPhotoUrls() != null) {
                for (String url : c.getPhotoUrls()) {
                    fileStorageService.delete(url);
                }
            }
        }
        loanRepository.delete(loan);
    }
    @Transactional(readOnly = true)
    public List<Loan> findByClientId(Long clientId) { return
loanRepository.findByClientId(clientId); }
    public long count() { return loanRepository.count(); }
    public BigDecimal getAverageLoanAmount() {
        Double avg = loanRepository.getAverageLoanAmount();
        return avg == null ? BigDecimal.ZERO : BigDecimal.valueOf(avg);
    }
    // среднее число дней от подачи заявки до решения менеджера
    public Double getAverageWaitingDays() {
        return loanRepository.findDecisionDates().stream()
                .mapToLong(row -> ChronoUnit.DAYS.between((LocalDate) row[0], (LocalDate)
row[1]))
                .average()
                .orElse(0.0);
    }
    public Map<String, Long> countByStatus() {
        List<Object[]> results = loanRepository.countByStatus();
        Map<String, Long> map = new HashMap<>();
        for (Object[] row : results) {
            map.put((String) row[0], ((Number) row[1]).longValue());
        }
        return map;
    }
    public Map<String, BigDecimal> sumByStatus() {
        List<Object[]> results = loanRepository.sumByStatus();
        Map<String, BigDecimal> map = new HashMap<>();
        for (Object[] row : results) {
            String status = (String) row[0];
            Object value = row[1];
            BigDecimal sum;
            if (value == null) {
                sum = BigDecimal.ZERO;
            } else if (value instanceof BigDecimal) {
                sum = (BigDecimal) value;
            } else if (value instanceof Number) {
                sum = BigDecimal.valueOf(((Number) value).doubleValue());
            } else {
                sum = BigDecimal.ZERO;
            }
            map.put(status, sum);
        }
        return map;
```

```
    }
    public List<Loan> searchLoans(String status, String search, String date) {
        LocalDate parsedDate = null;
        if (date != null && !date.isEmpty()) {
            try { parsedDate = LocalDate.parse(date); } catch (Exception ignored) {}
        }
        final LocalDate filterDate = parsedDate;
        // фильтр по статусу поисковой строке и дате
        return loanRepository.findAll().stream()
                .filter(l -> status == null || status.isEmpty() ||
status.equals(l.getStatus()))
                .filter(l -> {
                    if (search == null || search.isEmpty()) return true;
                    String s = search.toLowerCase().trim();
                    if (l.getLoanNumber() != null &&
l.getLoanNumber().toLowerCase().contains(s)) return true;
                    if (l.getClient() != null && l.getClient().getFullName() != null
                            && l.getClient().getFullName().toLowerCase().contains(s))
return true;
                    if (l.getClient() != null && l.getClient().getClientNumber() != null
                            && l.getClient().getClientNumber().toLowerCase().contains(s))
return true;
                    if (!l.getCollaterals().isEmpty()
                            && l.getCollaterals().get(0).getItemNumber() != null
                            &&
l.getCollaterals().get(0).getItemNumber().toLowerCase().contains(s)) return true;
                    return false;
                })
                .filter(l -> filterDate == null
                        || (l.getCreatedDate() != null &&
l.getCreatedDate().equals(filterDate)))
                .toList();
    }
    @Transactional
    public Loan createLoan(Loan loan, int months) {
        loan.setCreatedDate(LocalDate.now());
        loan.setDueDate(LocalDate.now().plusMonths(months));
        if (loan.getStatus() == null || loan.getStatus().isEmpty()) loan.setStatus("В
ожидании");
        return loanRepository.save(loan);
    }
    @Transactional
    public void closeLoan(Long id) {
        Loan loan = loanRepository.findById(id).orElse(null);
        if (loan != null) { loan.setStatus("Закрыт"); loanRepository.save(loan); }
    }
    @Transactional
    public void sellLoan(Long id) {
        Loan loan = loanRepository.findById(id).orElse(null);
        if (loan != null) { loan.setStatus("На продаже"); loanRepository.save(loan); }
    }
    @Transactional
    public void extendLoan(Long id, int months) {
        Loan loan = loanRepository.findById(id).orElse(null);
        if (loan != null) {
            // если дата возврата не задана - пляшу от текущей
            LocalDate base = loan.getDueDate() != null ? loan.getDueDate() :
LocalDate.now();
            loan.setDueDate(base.plusMonths(months));
            loanRepository.save(loan);
        }
    }
    @Transactional
    public void processOverdueLoans(NotificationService notificationService) {
        LocalDate today = LocalDate.now();
        for (Loan l : loanRepository.findAll()) {
            if (l.getDueDate() == null) continue;
            String itemName = getItemName(l);
            // напоминаю за 3 дня за 1 день и в день возврата
            if ("Выдан".equals(l.getStatus())) {
                long daysLeft = l.getDaysLeft();
```

```
                if (daysLeft == 3 || daysLeft == 1 || daysLeft == 0) {
                    String whenKey;
                    if (daysLeft == 0) whenKey = "сегодня последний день";
                    else if (daysLeft == 1) whenKey = "остался 1 день";
                    else whenKey = "осталось 3 дня";
                    if (l.getClient() != null && !reminderAlreadySent(l.getId(),
daysLeft)) {
                        notificationService.notify(l.getClient(), "LOAN_REMINDER",
"WARNING",
                                l.getLoanNumber(), itemName, whenKey,
                                l.getTotalToReturn().toPlainString(),
                                String.valueOf(l.getDueDate()));
                        loanHistoryService.log(l,
                                "Напоминание " + daysLeft + " дн. до срока возврата",
                                "SYSTEM");
                    }
                }
            }
            // срок вышел перевожу в просроченные
            if ("Выдан".equals(l.getStatus()) && l.getDueDate().isBefore(today)) {
                l.setStatus("Просрочен");
                loanRepository.save(l);
                if (!overdueAlreadySent(l.getId())) {
                    if (l.getClient() != null) {
                        notificationService.notify(l.getClient(), "LOAN_OVERDUE",
"DANGER",
                                l.getLoanNumber(), itemName,
String.valueOf(l.getDueDate()));
                    }
                    notificationService.notifyStaff("LOAN_OVERDUE_STAFF", "DANGER",
                            l.getLoanNumber(), itemName,
                            l.getClient() != null ? l.getClient().getFullName() : "-");
                    loanHistoryService.log(l, "Просрочен. Ожидает оплаты.", "SYSTEM");
                }
            }
            // если просрочка больше 30 дней (залог уходит на склад)
            if ("Просрочен".equals(l.getStatus())) {
                long daysOverdue = ChronoUnit.DAYS.between(l.getDueDate(), today);
                if (daysOverdue >= OVERDUE_TO_WAREHOUSE_DAYS &&
!movedToWarehouse(l.getId())) {
                    l.setStatus("На складе");
                    loanRepository.save(l);
                    if (l.getClient() != null) {
                        notificationService.notify(l.getClient(), "LOAN_ON_WAREHOUSE",
"DANGER",
                                l.getLoanNumber(), itemName,
                                String.valueOf(daysOverdue));
                    }
                    notificationService.notifyStaff("LOAN_ON_WAREHOUSE_STAFF", "DANGER",
                            l.getLoanNumber(), itemName,
                            l.getClient() != null ? l.getClient().getFullName() : "-");
                    loanHistoryService.log(l,
                            "Переведен на склад (просрочка " + daysOverdue + " дн.)",
                            "SYSTEM");
                }
            }
        }
    }
    // проверки чтобы не спамить уведомлениями
    private boolean overdueAlreadySent(Long loanId) {
        return loanHistoryRepository.findByLoanIdOrderByChangedAtDesc(loanId).stream()
                .anyMatch(h -> h.getAction() != null
                        && h.getAction().toLowerCase().startsWith("просрочен"));
    }
    private boolean reminderAlreadySent(Long loanId, long daysLeft) {
        String marker = "напоминание " + daysLeft;
        return loanHistoryRepository.findByLoanIdOrderByChangedAtDesc(loanId).stream()
                .anyMatch(h -> h.getAction() != null
                        && h.getAction().toLowerCase().contains(marker));
    }
    private boolean movedToWarehouse(Long loanId) {
```

```
        return loanHistoryRepository.findByLoanIdOrderByChangedAtDesc(loanId).stream()
                .anyMatch(h -> h.getAction() != null
                        && h.getAction().toLowerCase().contains("переведен на склад"));
    }
    @Transactional
    public boolean repayLoan(Long loanId, Long clientId, AccountService accountService) {
        Loan loan = loanRepository.findById(loanId).orElse(null);
        if (loan == null || loan.getClient() == null
                || !loan.getClient().getId().equals(clientId)) return false;
        if (!"Выдан".equals(loan.getStatus()) && !"Просрочен".equals(loan.getStatus()))
return false;
        try {
            accountService.withdraw(clientId, loan.getTotalToReturn(),
                    "Погашение займа " + loan.getLoanNumber());
        } catch (RuntimeException e) { return false; }
        loan.setStatus("Закрыт");
        loanRepository.save(loan);
        return true;
    }
    // беру название вещи из первого залога если есть
    public String getItemName(Loan loan) {
        if (loan == null) return "-";
        if (!loan.getCollaterals().isEmpty()
                && loan.getCollaterals().get(0).getName() != null
                && !loan.getCollaterals().get(0).getName().isBlank()) {
            return loan.getCollaterals().get(0).getName();
        }
        return "Товар";
    }
}
```

ПРИЛОЖЕНИЕ Г

Класс GlobalExceptionHandler

```
package com.lombard.controller;
@ControllerAdvice
public class GlobalExceptionHandler {
    private static final Logger log =
LoggerFactory.getLogger(GlobalExceptionHandler.class);
    // 404 - если не нашел страницу или статический ресурс
    @ExceptionHandler({NoHandlerFoundException.class, NoResourceFoundException.class})
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public String handleNotFound(Exception ex, Model model) {
        log.warn("404: {}", ex.getMessage());
        return "error/404";
    }
    // нет прав (сработал @PreAuthorize) - показываю страницу 403, а не "внутреннюю
ошибку"
    @ExceptionHandler(AccessDeniedException.class)
    @ResponseStatus(HttpStatus.FORBIDDEN)
    public String handleAccessDenied(AccessDeniedException ex) {
        log.warn("403: {}", ex.getMessage());
        return "error/403";
    }
    // ловлю ошибки целостности БД (например уникальные поля)
    @ExceptionHandler(DataIntegrityViolationException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public String handleDataIntegrity(DataIntegrityViolationException ex, Model model) {
        log.error("DataIntegrityViolation", ex);
        model.addAttribute("error", "Ошибка целостности данных");
        model.addAttribute("message", "Попытка сохранить некорректные данные. Проверьте
правильность ввода.");
        return "error";
    }
    // неверные аргументы (валидация, парсинг)
    @ExceptionHandler(IllegalArgumentException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
```

```
    public String handleIllegalArgument(IllegalArgumentException ex, Model model) {
        log.warn("IllegalArgument: {}", ex.getMessage());
        model.addAttribute("error", "Некорректный ввод данных");
        model.addAttribute("message", ex.getMessage());
        return "error";
    }
    // неправильное состояние например пытаюсь удалить то, что нельзя
    @ExceptionHandler(IllegalStateException.class)
    @ResponseStatus(HttpStatus.CONFLICT)
    public String handleIllegalState(IllegalStateException ex, Model model) {
        log.warn("IllegalState: {}", ex.getMessage());
        model.addAttribute("error", "Некорректное состояние");
        model.addAttribute("message", ex.getMessage());
        return "error";
    }
    // все остальное чтобы приложение не падало
    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public String handleException(Exception ex, Model model) {
        log.error("Unhandled exception", ex);
        model.addAttribute("error", "Внутренняя ошибка сервера");
        model.addAttribute("message", "Произошла непредвиденная ошибка. Попробуйте
позже.");
        return "error";
    }
}
```

ПРИЛОЖЕНИЕ Д

Класс Loan (Модель данных)

```
package com.lombard.model;
@Entity
@Table(name = "loans")
public class Loan {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(name = "loan_number", unique = true, length = 100)
    private String loanNumber;
    @NotNull(message = "Сумма не может быть пустой")
    @DecimalMin(value = "0.01", message = "Сумма должна быть больше 0")
    @Column(precision = 19, scale = 2)
    private BigDecimal amount;
    @Column(precision = 10, scale = 2)
    private BigDecimal rate; // ставка в процентах
    @Column(length = 50)
    private String status;
    private LocalDate createdDate; // когда создан
    private LocalDate dueDate; // вернуть до
    private LocalDate decisionDate; // когда менеджер принял решение
    @Column(name = "client_approved")
    private Boolean clientApproved;
    @Column(name = "agreed_amount", precision = 19, scale = 2)
    private BigDecimal agreedAmount;
    @ManyToOne
    @JoinColumn(name = "client_id")
    private Client client;
    @OneToMany(mappedBy = "loan", cascade = CascadeType.ALL, orphanRemoval = true) //
один заем может иметь несколько залогов
    private List<Collateral> collaterals = new ArrayList<>();
    @PrePersist
    public void prePersist() {
        if (createdDate == null) { // если дата создания не задана ставлю текущую
            createdDate = LocalDate.now();
        }
    }
    public Loan() {}
```

```
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getLoanNumber() { return loanNumber; }
    public void setLoanNumber(String loanNumber) { this.loanNumber = loanNumber; }
    public BigDecimal getAmount() { return amount; }
    public void setAmount(BigDecimal amount) { this.amount = amount; }
    public BigDecimal getRate() { return rate; }
    public void setRate(BigDecimal rate) { this.rate = rate; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public LocalDate getCreatedDate() { return createdDate; }
    public void setCreatedDate(LocalDate createdDate) { this.createdDate = createdDate; }
    public LocalDate getDueDate() { return dueDate; }
    public void setDueDate(LocalDate dueDate) { this.dueDate = dueDate; }
    public LocalDate getDecisionDate() { return decisionDate; }
    public void setDecisionDate(LocalDate decisionDate) { this.decisionDate =
decisionDate; }
    public Boolean getClientApproved() { return clientApproved; }
    public void setClientApproved(Boolean clientApproved) { this.clientApproved =
clientApproved; }
    public BigDecimal getAgreedAmount() { return agreedAmount; }
    public void setAgreedAmount(BigDecimal agreedAmount) { this.agreedAmount =
agreedAmount; }
    public Client getClient() { return client; }
    public void setClient(Client client) { this.client = client; }
    public List<Collateral> getCollaterals() { return collaterals; }
    public void setCollaterals(List<Collateral> collaterals) { this.collaterals =
collaterals; }
    // Логика расчетов и таймеров представлена в архиве с проектом.
}
```

ПРИЛОЖЕНИЕ Е

Файл application.properties

```
# Database settings
spring.datasource.url=${DB_URL:jdbc:postgresql://localhost:5432/lombard_db}
spring.datasource.username=${DB_USER:postgres}
spring.datasource.password=${DB_PASSWORD:1}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
# Thymeleaf
spring.thymeleaf.cache=false
spring.thymeleaf.mode=HTML
spring.thymeleaf.encoding=UTF-8
# Files
spring.servlet.multipart.enabled=true
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=50MB
# Uploads (collateral photos)
app.upload.dir=${UPLOAD_DIR:./uploads/}
# Logging
logging.level.org.springframework.boot.autoconfigure.orm.jpa=ERROR
logging.level.com.lombard=INFO
```